

Búsqueda Semántica de Objetos en Simulación con Razonamiento por Habitaciones

Estado del proyecto y plan de desarrollo

VLMaps + YOLOE + Habitat-Sim · HSSD como dataset principal

Extensión futura: ROS 1 Noetic + Toyota HSR

Mario García Blasco

22 de abril de 2026

Índice

1. Propósito del proyecto	3
1.1. Problema	3
1.2. Hipótesis	3
1.3. Tres niveles de conocimiento	3
1.4. Por qué un solo nivel no basta	3
1.5. Idea clave: razonar antes de moverse	4
1.6. De búsqueda plana (M2) a búsqueda jerárquica (room-aware)	4
2. Alcance y decisiones de diseño	5
2.1. HSSD como dataset principal	5
2.2. MP3D como soporte secundario	5
2.3. El razonamiento por habitaciones es la prioridad	5
2.4. Inspiración conceptual: MoMa-LLM	5
3. Estado actual del sistema	6
3.1. Resumen ejecutivo	6
3.2. Componentes completados	6
3.3. Tabla resumen de estado	8
3.4. Resumen honesto del estado	8
4. Arquitectura funcional actual	8
4.1. Pipeline de búsqueda M2	8
4.2. Arquitectura por capas de <code>tfg-sim</code>	9
4.3. Flujo completo del heatmap	9
4.4. Proveedor de habitaciones HSSD	10
4.5. Archivos clave del proyecto	10
5. Plan: cierre de la política room-aware	11
5.1. Visión general	11
5.2. Por qué Fase D es el punto de inflexión del proyecto	11
5.3. Fase D — Selector de habitación	12
5.4. Fase E — Selección de candidatos intra-habitación	12
5.5. Fase F — Acciones de alto nivel	13
5.6. Fase G — LLM estratégico sobre estado estructurado	13

5.7. Fase H — Métricas que demuestran la hipótesis	14
5.8. Orden de implementación	15
6. Fase posterior: baselines, evaluación y nuevos objetos	15
6.1. Baselines M1 y M3	15
6.2. Evaluación cuantitativa	16
6.3. Objetos pequeños no representados en el VLMap	16
7. Apartado extra: comparación baseline vs. executor	16
8. Extensión futura: ROS 1 Noetic + Toyota HSR	22
9. Diario de desarrollo	24

1. Propósito del proyecto

1.1. Problema

Dado un objetivo expresado en lenguaje natural —por ejemplo, “*encuentra la botella de agua*”, “*busca la taza*” o “*ve a la cocina y mira si hay un portátil en la mesa*”—, un robot simulado debe localizar el objeto de forma eficiente en un entorno doméstico interior.

El robot opera sobre escenas HSSD (*Habitat Synthetic Scenes Dataset*) renderizadas con Habitat-Sim. La escena principal es **102344280**, una vivienda de una planta con 15 habitaciones anotadas ($\sim 447\text{ m}^2$). El robot no conoce la posición exacta de los objetos: dispone únicamente de un mapa semántico pre-construido (VLMMap), de las anotaciones nativas de habitaciones de HSSD y de la imagen RGB de su cámara.

El objetivo del proyecto no es ampliar infraestructura genérica de simulación ni acumular datasets, sino construir un sistema de búsqueda que entienda la estructura semántica de la vivienda. El robot debe razonar sobre dónde tiene sentido buscar antes de lanzarse a inspeccionar candidatos arbitrarios, y debe ser capaz de encontrar incluso objetos que ni siquiera están explícitamente representados en el mapa semántico (botellas, tazas, portátiles, cajas, etc.).

1.2. Hipótesis

La hipótesis central del TFG es la siguiente:

Un robot que razona explícitamente sobre la estructura de la casa —primero por habitación, después por mueble o zona y finalmente por verificación visual del objeto— encuentra objetos domésticos de forma más eficiente que un sistema que busca sobre un heatmap semántico global o que se apoya en un único nivel de información.

«Más eficiente» se traduce en métricas concretas: menos habitaciones visitadas antes del éxito, menos candidatos inspeccionados, mayor proporción de aciertos al primer intento y caminos más cortos. Estas métricas se introducen formalmente en la Sección 5.7.

1.3. Tres niveles de conocimiento

El sistema combina tres niveles semánticos jerárquicos. Cada uno aporta información que ningún otro puede sustituir:

Nivel	Fuente	Qué aporta
1. Habitación	<code>semantic_config.json</code> de HSSD	«Esto es la cocina; aquello es el baño». Polígonos de planta con etiqueta.
2. Mueble / zona	VLMMap (36 categorías HSSD)	«Aquí hay una encimera; ahí una mesa». Localización aproximada de las superficies sobre las que típicamente reposan los objetos.
3. Objeto específico	YOLOE (vocabulario abierto)	«Sobre esa encimera hay una botella». Verificación visual final con la cámara del robot.

1.4. Por qué un solo nivel no basta

Cualquiera de los tres niveles utilizado de forma aislada produce un sistema incompleto:

- **VLMMap por sí solo no encuentra objetos pequeños.** Las categorías indexadas en el VLMMap son muebles y superficies estáticas (`table`, `counter`, `sofa`, `bed`, `cabinet`, etc.). Objetos de interés cotidianos como `bottle`, `cup`, `box` o `laptop` no aparecen en el VLMMap,

porque CLIP los confunde fácilmente con los muebles que los soportan o porque su huella es demasiado pequeña en la proyección 2D. Sin una capa adicional, el sistema no tiene forma de *encontrar* esos objetos: como mucho, puede localizar el mueble donde típicamente residen.

- **YOLOE por sí solo no sabe a dónde ir.** YOLOE es un detector de vocabulario abierto que opera sobre la imagen actual. No tiene memoria espacial ni mapa, por lo que sin un mecanismo que lo lleve frente a las zonas plausibles de la casa, su trabajo se reduce a inspeccionar el campo de visión actual. Una búsqueda basada exclusivamente en YOLOE degenera en exploración ciega.
- **El razonamiento por habitaciones es el pegamento que falta.** Saber que una botella vive típicamente en la cocina permite al robot saltarse cuatro habitaciones que no aportan información. Saber que dentro de la cocina debe mirar primero encimeras y mesas reduce drásticamente el número de candidatos a verificar con YOLOE. El razonamiento por habitaciones convierte una búsqueda exhaustiva en una búsqueda dirigida.

1.5. Idea clave: razonar antes de moverse

La política de búsqueda del sistema sigue siempre la misma estructura jerárquica:

1. Identificar las habitaciones de la escena y construir un *prior* sobre cuál es la más probable para el objeto buscado.
2. Traducir el objeto a un conjunto de categorías VLMap plausibles, de forma que el sistema sepa qué muebles o zonas inspeccionar dentro de la habitación elegida (por ejemplo `laptop` → `[table, counter, shelving]`).
3. Localizar instancias de esas categorías *dentro de la habitación seleccionada*, no globalmente.
4. Navegar al candidato más prometedor.
5. Verificar visualmente con YOLOE si el objeto está allí.
6. Si la verificación falla, continuar con el siguiente candidato dentro de la misma habitación; si se agotan, pasar a la siguiente habitación más probable. Si la verificación tiene éxito, declarar el objeto **ENCONTRADO**.

Este flujo es la traducción operativa de la hipótesis. La diferencia decisiva con respecto a un sistema de búsqueda plana está en el orden: se decide *en qué habitación* buscar antes de decidir *qué candidato* inspeccionar.

1.6. De búsqueda plana (M2) a búsqueda jerárquica (room-aware)

El sistema dispone hoy de la infraestructura necesaria para ambos estilos de búsqueda. La diferencia conceptual entre ellos es la siguiente:

- **Búsqueda plana (M2).** Para cada categoría se construye un heatmap global sobre todo el mapa, se extraen componentes conexos, se ordenan por una métrica de calidad y se navega al primero. La habitación a la que pertenece el candidato es información incidental: no participa en la decisión. El robot puede cruzar la casa entera para ir a un candidato ruidoso situado en una habitación irrelevante.
- **Búsqueda jerárquica (room-aware).** La decisión se descompone en dos niveles. Primero se elige la habitación con mayor utilidad esperada para la query (combinando priors LLM, evidencia del heatmap, distancia y exploración previa). Después se ejecuta el pipeline M2, pero restringiendo los candidatos a los que caen dentro de esa habitación. Si la habitación elegida se agota sin éxito, se pasa a la siguiente más probable.

El sistema room-aware no reemplaza al pipeline M2: lo *encapsula*. Toda la maquinaria de heatmaps,

planificación y verificación YOLOE sigue siendo relevante; la diferencia es que ahora actúa dentro de la habitación correcta y en el orden correcto. La Sección 5 desarrolla el plan concreto para cerrar este bucle.

2. Alcance y decisiones de diseño

2.1. HSSD como dataset principal

HSSD es un conjunto de escenas de interiores sintéticas de alta calidad desarrolladas para Habitat-Sim. Su elección como dataset principal responde a tres motivos directamente alineados con la hipótesis del proyecto:

- **Habitaciones anotadas nativamente.** Cada escena incluye un fichero `semantics/scenes/<id>.semant` con polígonos de planta y nombres de habitación. La clase `SemanticSceneRoomProvider` los rasteriza directamente sobre el grid VLMMap sin intervención manual. El razonamiento por habitaciones descansa sobre estas anotaciones, por lo que su disponibilidad nativa es decisiva.
- **Geometría limpia e iluminación controlada.** Las escenas sintéticas facilitan la verificación visual con YOLOE y eliminan artefactos de reconstrucción fotogramétrica que afectan a otros datasets.
- **Alineación con el objetivo del proyecto.** Sin anotaciones de habitación fiables, el proyecto se vería forzado a etiquetar manualmente cada escena, desplazando esfuerzo del problema central (razonamiento jerárquico) a un problema instrumental (anotación).

La escena principal es **102344280**: una vivienda de una planta con 15 habitaciones (`kitchen`, `dining room`, `bedroom`×3, `bathroom`×3, `hallway`, `entryway`, `closet`×3, `tv room`).

2.2. MP3D como soporte secundario

Toda la funcionalidad MP3D existente se mantiene intacta: el sistema sigue funcionando sobre escenas MP3D y existe un `LabelMeRoomProvider` listo para usar si se ejecuta el etiquetado manual. Sin embargo, MP3D no condiciona la hoja de ruta actual:

- El etiquetado LabelMe de las 3 escenas MP3D queda **diferido** como tarea secundaria.
- La construcción de VLMaps para escenas MP3D adicionales queda diferida.
- Las limitaciones visuales de MP3D (texturas ruidosas, reconstrucciones imperfectas) perjudican la verificación YOLOE y no son representativas de la capacidad real del método.

2.3. El razonamiento por habitaciones es la prioridad

La siguiente fase del proyecto es que el robot pueda recibir instrucciones del estilo “*encuentra la taza*”, “*ve a la cocina y mira si hay una botella en la encimera*” o “*busca el portátil*”, y resuelva la búsqueda usando los tres niveles semánticos descritos en la Sección 1.3.

Esta fase es prioritaria frente a la inserción de nuevos objetos en simulación, los baselines M1/M3 y la evaluación cuantitativa amplia. Sin razonamiento por habitaciones operativo, el sistema no puede demostrar la ventaja del nivel más alto de la hipótesis y, por tanto, no puede cerrar la contribución del TFG.

2.4. Inspiración conceptual: MoMa-LLM

El paper *Language-Grounded Dynamic Scene Graphs for Interactive Object Search with Mobile Manipulation* (Honerkamp et al., RSS SemRob 2024) propone usar un LLM para razonar sobre un

grafo de escena estructurado por habitaciones. Este proyecto adopta esa idea conceptualmente: habitaciones como entidades explícitas, estado estructurado para el LLM, historial compacto de búsqueda y selección iterativa de la siguiente habitación.

No se utiliza ningún código del repositorio MoMa-LLM. La implementación es completamente propia. La diferencia principal es que MoMa-LLM construye el grafo dinámicamente durante la exploración, mientras que este proyecto usa un VLMap pre-construido y habitaciones pre-anotadas (HSSD), lo que permite una evaluación más controlada y reproducible.

3. Estado actual del sistema

3.1. Resumen ejecutivo

A día de hoy el sistema dispone de toda la infraestructura, todos los componentes semánticos y la mayor parte de la maquinaria room-aware. El pipeline M2 (navegación interactiva sobre HSSD con verificación YOLOE) funciona de extremo a extremo, y las fases preparatorias del razonamiento por habitaciones (A, B, C de la Sección 5) están completas.

Lo que aún no está cerrado es el **bucle de decisión** que convierte ese material en una política de búsqueda jerárquica dominante: actualmente el sistema usa los priors de habitación como información auxiliar, pero la selección final del candidato sigue ocurriendo de forma esencialmente plana sobre el heatmap. Esa es la pieza pendiente, y es exactamente la diferencia decisiva que permitirá demostrar la hipótesis central del TFG.

3.2. Componentes completados

Infraestructura

- ✓ Docker con CUDA 12.1 + Habitat-Sim 0.3.1 + conda (Python 3.9).
- ✓ 3 escenas MP3D con VLMaps contruidos: `gTV8FGcVJC9_1`, `jh4fc5c5qoQ_1`, `mJXqzFtmKg4_1`.
- ✓ Escena HSSD integrada: **102344280** (15 habitaciones, $\sim 447\text{m}^2$), VLMap construido e indexado.
- ✓ Menú interactivo en Docker (`menu.sh`) para toda la pipeline.
- ✓ Submódulo `third_party/vlmaps` (rama `tfg-modifications`).
- ✓ Repositorio limpio en GitHub (eliminado el binario de 571 MB del historial con `git-filter-repo`).

Pipeline M2 (búsqueda plana funcional)

- ✓ Flujo completo instrucción $\rightarrow$ LLM $\rightarrow$ selección de goal $\rightarrow$ planificación $\rightarrow$ navegación live con visualización.
- ✓ Planificador A* *clearance-aware* sobre el navmesh de Habitat. La colisión física se delega a la propia navmesh (*sliding*); un watchdog de no-progreso aborta si el robot deja de avanzar (ver diario del 16 y 20 de abril).
- ✓ `plan_path_only()` muestra el camino antes de ejecutarlo (línea roja, 800 ms de previsualización).
- ✓ Multi-goal retry: hasta 8 candidatos por categoría con descarte por seguridad y por intentos previos.

Componentes semánticos

- ✓ **Mapper LLM:** GPT-4o-mini traduce el objeto a categorías VLMap probables y a habitaciones plausibles. Distingue queries directas (`sofa` ∈ categorías HSSD) e indirectas (`laptop` ∉ categorías HSSD). Ejemplo: `laptop` → {`categories`: [`table`, `counter`, `shelving`], `rooms`: [`office`, `bedroom`]}.
- ✓ **YOLOE como verificador:** worker persistente, protocolo stdout estable, scan 360° asíncrono, visual servoing iterativo, caché global por target. Verificación en tres etapas (stand-off, scan, alternativa). 15 categorías incompatibles (`wall`, `floor`, `ceiling`, ...) saltan la verificación.
- ✓ **Heatmap semántico** con postproceso de calidad por componente, filtros de coherencia, área y pared, y fórmula de *quality* (Sección 4.3).

Capa room-aware ya disponible (Fases A, B, C, D)

- ✓ **Fase A — capa de habitaciones HSSD.** `SemanticSceneRoomProvider` lee el `semantic_config.json`, transforma los polígonos de Habitat al grid VLMap y proporciona `get_room_at_cell(row,col)`, `get_room_centroid(name)` y `list_rooms()`.
- ✓ **Fase B — estado estructurado.** `RoomState` registra para cada habitación: centroide, celdas totales y libres, ratio explorado, objetos vistos, candidatos probados/confirmados, relevancia para el target actual. `SearchState` agrega todos los `RoomState`, mantiene el historial de visitas y la habitación actual.
- ✓ **Fase C — priors objeto → habitación.** Fusión ponderada de tres fuentes (LLM, tabla manual de ~35 entradas, evidencia de co-ocurrencia) en `compute_room_priors()`. El resultado normalizado se almacena en `RoomState.target_relevance` y se imprime como traza al inicio de cada búsqueda.
- ✓ **Fase D — selector de habitación.** Antes del ranking final de candidatos, el sistema calcula un `room_score` por habitación combinando prior semántico, evidencia del heatmap, coste BFS desde la posición actual y penalización por intentos fallidos. La habitación ganadora restringe el conjunto de candidatos que pasa al ranking fino.

Soporte experimental

- ✓ `place_objects.py`: define objetos 3D personalizados (.glb) vía JSON. Los objetos quedan visibles para YOLOE pero ausentes del VLMap, lo que permite evaluar directamente el caso de uso *objetos pequeños no representados en el mapa semántico*.

3.3. Tabla resumen de estado

Componente	Estado
Infraestructura Docker + Habitat + VLMaps	Completado
Pipeline M2 (planificación + ejecución)	Completado
Mapper LLM (objeto → categorías + rooms)	Completado
YOLOE persistente + scan + visual servoing	Completado
Filtros de calidad y heatmap	Completado
Integración HSSD completa	Completado
Inserción de objetos personalizados	Completado
Fase A: capa de habitaciones HSSD	Completado
Fase B: RoomState + SearchState	Completado
Fase C: priors objeto → habitación	Completado
Fase D: selector de habitación	Completado
Fase E: candidatos intra-habitación	Completado
Fase F: acciones de alto nivel	Completado
Fase G: LLM estratégico con estado	Parcial
Fase H: métricas room-aware	Parcial
Apartado extra: baseline vs. executor	Parcial
Baselines M1, M3	Posterior
Evaluación cuantitativa amplia	Posterior
Inserción extensiva de objetos pequeños	Posterior
Extensión ROS 1 + Toyota HSR	Futura

3.4. Resumen honesto del estado

El sistema dispone ya de los tres pilares semánticos (habitación, mueble, objeto) y de toda la infraestructura para usarlos. Las tres decisiones jerárquicas básicas están integradas: el selector de habitación (Fase D) decide qué habitación merece inspección, la selección intra-habitación (Fase E) restringe los candidatos al subconjunto relevante, y la capa de acciones de alto nivel (Fase F) expone esas operaciones como un vocabulario discreto (**go_to_room**, **explore_room**, **inspect_candidate**, **verify_target**, **done**) con serialización JSON, listo para ser producido por un LLM estratégico.

Lo que falta es cerrar la cabeza de la política room-aware: conectar un LLM estratégico que emita esas acciones a partir del estado estructurado (Fase G) y construir la batería de métricas que demuestre cuantitativamente la mejora frente a la línea base M2 (Fase H).

4. Arquitectura funcional actual

4.1. Pipeline de búsqueda M2

El pipeline M2 funciona de extremo a extremo y constituye la base sobre la que operará la política room-aware. Su flujo es:

1. **Instrucción.** El usuario introduce una query en lenguaje natural.
2. **Parsing LLM.** `parse_object_goal_instruction()` (GPT-4o-mini) extrae las categorías semánticas implicadas.
3. **Heatmap semántico.** Para cada categoría, `compute_heatmap()` genera un mapa de probabilidad 2D a partir de los scores CLIP del VLMap.
4. **Postproceso.** `postprocess_heatmap()` filtra ruido, calcula componentes conexos y asigna

un score de calidad a cada uno.

5. **Selección de goal.** Se elige el candidato con mayor calidad efectiva (incluyendo bonus de proximidad y bonus por habitación local) y se le añade un anillo de aproximación seguro alrededor del centroide.
6. **Planificación.** `plan_path_only()` calcula el camino con A* clearance-aware sin ejecutarlo y permite previsualizarlo.
7. **Ejecución.** El *path follower* con *lookahead* recorre el camino sobre el navmesh. La colisión se delega a Habitat (sliding sobre el navmesh) y un watchdog de no-progreso aborta si el robot deja de avanzar.
8. **Verificación YOLOE.** Tres stages: standoff a 0,25 m, scan 360° asíncrono y componente alternativa del heatmap.
9. **Retry.** Si falla, se prueba el siguiente candidato (hasta 8); si se agota la categoría, se pasa a la siguiente.

4.2. Arquitectura por capas de tfg-sim

La organización actual del sistema en `tfg-sim` puede resumirse como una arquitectura de tres capas. La idea clave es que la semántica y la estrategia viven por encima del backend de simulación, de modo que la futura migración a ROS/HSR sólo tenga que sustituir la capa inferior sin reescribir la lógica de decisión.

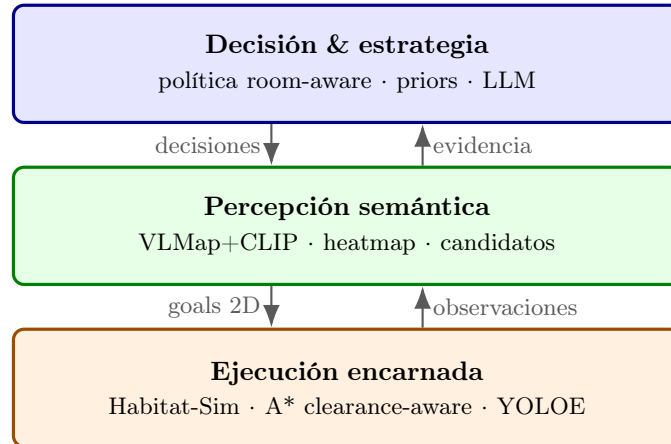


Figura 1: Arquitectura por capas de `tfg-sim`. La capa inferior es la única que cambia al migrar a ROS/HSR; la semántica y la estrategia se preservan.

4.3. Flujo completo del heatmap

Paso 1 — Scores CLIP por vóxel

Cada vóxel del mapa 3D tiene un vector de scores CLIP de dimensión $|\mathcal{C}|$. Para una query dada, se extrae el score $s_i \in [0, 1]$ de cada vóxel i para la categoría objetivo. Solo se retienen vóxeles con argmax en esa categoría y $s_i > 0,3$.

Paso 2 — Proyección 2D (max-pool por columna)

Los scores 3D se proyectan sobre el plano del suelo tomando el máximo por celda (r, c) . Resultado: heatmap flotante $gs \times gs$ (por defecto $gs = 960$).

Paso 3 — Difusión por distancia

Difusión basada en `distance_transform_edt`: caída lineal $\times 0,3$ por celda. Celdas con `score < 0,15` se anulan (franja ~ 3 celdas = 15 cm).

Paso 4 — Suavizado Gaussiano

Gaussian blur 5×5 , $\sigma = 1,0$ para suprimir ruido de alta frecuencia.

Paso 5 — Umbral relativo y componentes conectados

Binarización con $t = 0,5 \times \text{máx}(\text{heatmap suavizado})$. Componentes conexos (8-conectividad). Para cada componente: área, score máximo, score medio, suma de scores, bounding box, centroide.

Paso 6 — Scoring interno y filtro relativo

$$\text{score}(r) = \text{mean_val}(r) \cdot \log(1 + \text{area}(r))$$

Se descartan componentes con $\text{score}(r) < 0,2 \cdot \text{máx}_j \text{score}(j)$.

Paso 7 — Fórmula de calidad

Combinación normalizada de soporte semántico real (`sum_val`), tamaño útil del componente, score CLIP medio y densidad. La formulación enfatiza el soporte real para evitar que activaciones pequeñas pero densas desplacen a regiones grandes y representativas (ver diario del 20 de abril).

Paso 8 — Selección del goal

A partir del centroide del componente ganador se busca un *anillo de aproximación seguro*: posiciones a 8–25 celdas del centroide, navegables y con holgura suficiente. La meta de navegación es la mejor posición de ese anillo, no el centroide en sí, lo que evita rutas que terminarían dentro de un mueble.

4.4. Proveedor de habitaciones HSSD

La clase `SemanticSceneRoomProvider` (en `room_provider.py`) lee el fichero `semantics/scenes/<id>.semantic` de cada escena HSSD y expone:

- `build(config_path, gs, hab_to_grid)`: lee `region_annotations`, convierte cada polígono de Habitat world a coordenadas del grid VLMaP mediante `hab_to_grid`, y rasteriza con test punto-en-polígono. Resultado: `_region_grid[gs, gs]` (int32, 0 = sin etiquetar).
- `get_room_at_cell(row, col)` → nombre de la habitación o `None`.
- `get_room_centroid(room_name)` → (row, col) del centroide de la habitación.
- `list_rooms()` → lista de nombres de todas las habitaciones.

Cada región almacena: `id, name, category, centroid [r, c], poly_xz, floor_height`.

4.5. Archivos clave del proyecto

Archivo	Rol
application/interactive_object_nav.py	Pipeline de navegación principal
vlmaps/robot/habitat_lang_robot.py	Robot + planificador + <code>plan_path_only</code>
vlmaps/utils/llm_utils.py	Mapper LLM: objeto → categorías + habitaciones
vlmaps/utils/yoloe_utils.py	YoloeSession: worker persistente, sync/async
vlmaps/utils/_yoloe_persistent_worker.py	Subprocess YOLOE aislado
vlmaps/utils/room_provider.py	SemanticSceneRoomProvider (HSSD nativo)
vlmaps/utils/room_priors.py	Priors objeto → habitación (Fase C)
vlmaps/utils/search_state.py	RoomState y SearchState (Fase B)
vlmaps/utils/navigation_utils.py	A* clearance-aware + <code>plan_clearance_aware_astar</code>
vlmaps/navigator/navigator.py	Adaptador del planificador clearance-aware
config/object_goal_navigation_cfg.yaml	Config Hydra principal
docker/menu.sh	Menú interactivo Docker

5. Plan: cierre de la política room-aware

5.1. Visión general

El sistema actual ya dispone de los tres niveles semánticos por separado —**habitación** (HSSD), **mueble/zona** (VLMaP) y **objeto** (YOLOE)— y de la maquinaria de estado por habitación (Fase B) y de los priors objeto → habitación (Fase C). Lo que aún no existe es el *módulo de decisión* que use esa información para dirigir la búsqueda de forma jerárquica.

El plan se divide en cinco fases incrementales (D–H). Cada fase aporta una capacidad concreta sin romper la anterior, y el conjunto convierte el sistema de búsqueda plana actual en un buscador room-aware completo y evaluable:

Fase	Aportación	Función en la hipótesis
D	Selector de habitación	Decide <i>dónde</i> buscar
E	Selección de candidatos intra-habitación	Decide <i>qué</i> inspeccionar
F	Acciones de alto nivel	Estructura la política
G	LLM estratégico sobre estado estructurado	Razona sobre la búsqueda
H	Métricas room-aware	Demuestra cuantitativamente la mejora

5.2. Por qué Fase D es el punto de inflexión del proyecto

Sin Fase D, el sistema puede tener priors de habitación, anotaciones, estado estructurado y todo el material semántico necesario, y aun así seguir comportándose como una búsqueda global: el flujo de control desemboca siempre en `select_best_candidate()` sobre el heatmap, donde la habitación es información incidental.

Con Fase D el flujo se invierte. La primera decisión deja de ser «¿cuál es el mejor componente del heatmap?» y pasa a ser «¿cuál es la mejor habitación para buscar este objeto ahora?». Solo cuando esa pregunta tiene respuesta se baja al nivel de candidatos.

Es el cambio mínimo que convierte el sistema en algo cualitativamente distinto de la línea base M2 y, por tanto, en algo que el TFG puede defender como contribución experimental. Las fases E, F y G refinan progresivamente esa política: E cierra la restricción espacial sobre los candidatos, F la formaliza como vocabulario de acciones y G permite que un LLM la dirija. La fase H proporciona la batería de métricas con la que se demuestra el valor del enfoque. Todas dependen de que exista la decisión por habitación.

5.3. Fase D — Selector de habitación

Objetivo. Que el sistema decida *primero* en qué habitación buscar y solo después qué candidato inspeccionar.

Idea. En cada ciclo de búsqueda, antes de elegir un componente del heatmap, se calcula una puntuación por habitación que combina: (i) el prior semántico para el objeto buscado (Fase C), (ii) la evidencia visual del heatmap dentro de la habitación, (iii) lo poco que se ha explorado, (iv) el coste de llegar y (v) la penalización acumulada por intentos fallidos previos:

$$\text{room_score}(r) = w_1 \text{prior}(r) + w_2 \text{evidence}(r) + w_3 \text{unexplored}(r) - w_4 \text{cost}(r) - w_5 \text{penalty}(r)$$

Significado de cada término:

- $\text{prior}(r)$: `RoomState.target_relevance` (Fase C), normalizado.
- $\text{evidence}(r)$: suma de la *quality* de los componentes del heatmap cuyo centroide cae en r (función ya disponible: `compute_heatmap_room_evidence()`).
- $\text{unexplored}(r) = 1 - \text{explored_ratio}(r)$.
- $\text{cost}(r)$: distancia BFS desde la posición actual del robot al centroide de r , normalizada al diámetro del mapa.
- $\text{penalty}(r)$: penalización por candidatos probados sin éxito previamente en r , leída de `RoomState`.

Pesos iniciales sugeridos: $w_1 = 0,30$, $w_2 = 0,25$, $w_3 = 0,20$, $w_4 = 0,15$, $w_5 = 0,10$, ajustables empíricamente.

Integración. Una nueva función `select_best_room(search_state, robot_pos, heatmap_evidence, obs_map)` devuelve el nombre de la habitación ganadora y se invoca al inicio de cada ciclo de búsqueda, antes de `select_best_candidate()`.

Lo que cambia operativamente. Esta fase es la que convierte el razonamiento por habitaciones en una decisión real del robot. Hasta ahora los priors de habitación se calculan, se imprimen y se usan como bonus dentro de la selección de candidato; con Fase D pasan a ser la primera decisión del ciclo y restringen el universo de candidatos que se consideran a continuación.

Esfuerzo estimado: 1–2 sesiones.

5.4. Fase E — Selección de candidatos intra-habitación

Objetivo. Una vez decidida la habitación, ejecutar la búsqueda detallada *dentro* de ella sin reescribir la pipeline existente.

Idea. La pipeline M2 sigue siendo la herramienta correcta para inspeccionar candidatos individuales: heatmap $\rightarrow$ goal $\rightarrow$ planificación $\rightarrow$ ejecución $\rightarrow$ YOLOE. Lo único que cambia es el *universo* de candidatos: ya no son los componentes globales del heatmap, sino los que caen dentro de la habitación elegida por Fase D.

1. Filtrar `kept_components` a aquellos cuyo centroide cae dentro de la habitación seleccionada (`get_room_at_cell()`).
2. Ordenar los candidatos filtrados por la *quality* ya implementada, posiblemente con bonus de proximidad.
3. Ejecutar la pipeline M2 sobre el mejor candidato.

4. Si no hay candidatos sólidos en la habitación, explorar el frontier más cercano dentro de ella (Fase F).
5. Si la query es de ubicación conocida (“*ve a la cocina*”), navegar directamente a un goal interior seguro de la habitación (mecanismo ya existente).

Esto **no descarta la pipeline M2**: la encapsula dentro de un bucle exterior dirigido por habitación. La diferencia operativa con respecto al sistema actual es que el mismo M2 se ejecuta sobre un subconjunto reducido y relevante de candidatos, en lugar de sobre el ranking global.

Esfuerzo estimado: 1 sesión.

5.5. Fase F — Acciones de alto nivel

Objetivo. Hacer explícito el vocabulario de acciones del robot, de forma que la política room-aware pueda manipularlo como tal y no como una maraña de llamadas de bajo nivel.

Acción	Descripción
<code>go_to_room(room)</code>	Navegar a un goal interior seguro de la habitación indicada
<code>explore_room(room)</code>	Cobertura sistemática de celdas no visitadas dentro de la habitación
<code>inspect_candidate(room, cand)</code>	Aproximación a un candidato concreto + verificación YOLOE
<code>verify_target</code>	Scan 360° + visual servoing en la posición actual
<code>done</code>	Objetivo encontrado, fin de la búsqueda

`go_to_room` e `inspect_candidate` se apoyan en la navegación existente. `explore_room` es nuevo: genera waypoints que cubran las celdas navegables no visitadas. Esta capa convierte la política room-aware en una secuencia de decisiones discretas que un LLM (Fase G) puede leer y producir de forma natural.

Estado. **Implementado** en el módulo `vlmaps/policy/` (commit del 20.04). Contiene:

- `policy/actions.py`: `ActionType` (enum de cinco valores), `Action` (dataclass inmutable validada en `__post_init__`) y dos helpers `action_to_json`/`parse_action_json` para el intercambio con el LLM (Fase G). Se publica también `ACTION_SCHEMA_PROMPT`, fragmento listo para incrustar en el *system prompt*.
- `policy/frontier.py`: `find_frontier_in_room()` elige una celda navegable dentro de una habitación, alejada de las celdas ya visitadas (parámetros `visited_radius_cells` y `min_clearance_cells`). Resuelve el componente nuevo de `explore_room`.
- Ampliación aditiva de `SearchState`: `action_log`, `visited_cells` y los métodos `record_action`, `record_visited_cell`, `recent_actions` (sin tocar la lógica existente, conforme al criterio *add-only* del repositorio).
- Suite de tests `tests/test_phase_f_actions.py` (15 casos): validación de cada `ActionType`, *round-trip* JSON, log de acciones, y *frontier picking* con/sin trail previa. Toda la batería pasa sin Habitat ni dependencias pesadas.

La capa es *aditiva*: el bucle interactivo actual sigue funcionando sin cambios, y la Fase G podrá enchufar el ejecutor que despache cada `Action` sobre las funciones inline ya validadas (`navigate_to_room_stage`, `select_best_candidate`, `scan_360_and_verify`, `fine_visual_center`).

5.6. Fase G — LLM estratégico sobre estado estructurado

Objetivo. Que el LLM decida *estrategia de búsqueda*, no trayectoria.

El LLM no recibe contexto bruto ni controla la navegación de bajo nivel: recibe un *estado estructurado y compacto* y devuelve la habitación que se debe visitar y la acción de alto nivel a ejecutar. La puntuación de Fase D queda como *fallback* numérico cuando el LLM no responda con claridad o como validación cruzada.

Ejemplo de prompt interno:

Task: find sink

Current room: hallway

Known rooms:

- bathroom: objects [mirror, toilet, shower], explored 80%,
1 candidate tried (vanity-zone: YOLOE no confirmation)
- kitchen: objects [counter, cabinet, table], explored 60%,
0 candidates tried
- living_room: objects [sofa, table, shelf], explored 90%

Unexplored zones:

- bathroom: near rear corner
- kitchen: near counter area

Which room should be searched next, and which category or zone inside it should be prioritized?

El historial enviado al LLM es compacto: las últimas 5 acciones, las habitaciones exploradas, los candidatos probados y el motivo de fallo (no detectado, heatmap débil, navegación fallida). Esto evita inflar el prompt y mantiene el coste y la latencia bajo control.

El detalle de cómo se integra esa decisión en el sistema sin tocar el pipeline actual — entrypoint nuevo, ejecutor de acciones y comparación de heatmaps — queda recogido en el apartado extra previo a la migración a ROS.

Estado. Implementación inicial lista. Se ha añadido un módulo nuevo `vlmaps/policy/strategic_policy.py` que construye un *snapshot* estructurado del episodio (habitación actual, prior/evidence por habitación, shortlist de candidatos, frontiers sugeridos y últimas acciones), intenta decidir la *siguiente* acción mediante LLM (`gpt-4o-mini`) y valida la salida contra el vocabulario de Fase F. Si el LLM falla, devuelve JSON inválido o propone una acción inconsistente, la decisión cae automáticamente al *fallback* heurístico. El `interactive_object_nav_executor.py` deja así de preplanificar una lista fija de acciones y pasa a operar en un bucle de `choose_next_action()` → `execute_action()`, con modo seleccionable `heuristic/hybrid/llm` desde el menú del contenedor. Queda pendiente la validación sistemática en runtime y el ajuste fino de la política, pero la transición arquitectónica ya está materializada.

Esfuerzo restante estimado: 1–2 sesiones.

5.7. Fase H — Métricas que demuestran la hipótesis

Objetivo. Medir cuantitativamente si el razonamiento por habitaciones mejora la búsqueda. Esta fase cierra el bucle del TFG: sin ella, las fases D–G son una propuesta arquitectónica, pero no una contribución demostrada.

Métricas estándar de navegación semántica:

- **SR** (*Success Rate*): fracción de episodios en los que YOLOE confirma el objetivo.
- **SPL** (*Success weighted by Path Length*): SR ponderado por la eficiencia del camino respecto

al óptimo.

- **Wrong Visits:** número de candidatos visitados sin detección positiva antes del éxito.
- **Time to Find:** pasos de simulación hasta la confirmación.

Métricas específicas de razonamiento por habitaciones (las que separan M2 de room-aware):

- **CFR** (*Correct First Room*): ¿fue la primera habitación visitada la que contenía el objetivo?
- **CT2R** (*Correct Top-2 Rooms*): ¿estaba la habitación correcta entre las dos primeras visitadas?
- **Rooms Before Success:** número de habitaciones visitadas antes de encontrar el objeto.
- **Inspections Before Success:** número de candidatos inspeccionados antes del éxito.

CFR, CT2R, *Rooms Before Success* e *Inspections Before Success* son las métricas que diferencian directamente un sistema room-aware de un sistema plano. Si el razonamiento por habitaciones aporta valor, debe reflejarse en ellas.

Esfuerzo estimado: 1 sesión.

5.8. Orden de implementación

Fase	Descripción	Depende de	Esfuerzo
A	Capa de habitaciones HSSD	Infraestructura	Hecho
B	RoomState + SearchState	A	Hecho
C	Priors objeto $\rightarrow$ habitación	Mapper LLM	Hecho
D	Selector de habitación	B + C	Hecho
E	Candidatos intra-habitación	D + pipeline M2	Hecho
F	Acciones de alto nivel	E	Hecho
G	LLM estratégico con estado	B + C + F	Parcial
H	Métricas de evaluación	F	1 sesión
Esfuerzo restante estimado			2 sesiones

D y E son el cambio operativo de comportamiento: tras estas dos fases el robot ya busca de forma jerárquica. F y G estructuran y delegan esa política. H la evalúa y permite contrastarla con la línea base M2. Esa secuencia es el camino mínimo y suficiente para cerrar la contribución del TFG.

6. Fase posterior: baselines, evaluación y nuevos objetos

Una vez consolidada la política room-aware (fases D–H), el proyecto entra en la fase de evaluación amplia y de ampliación del catálogo de objetos buscados.

6.1. Baselines M1 y M3

- **M1 — Navegación aleatoria:** el robot elige celdas navegables al azar como goals. Sirve como cota inferior de rendimiento.
- **M3 — Ranking global por calidad:** candidatos ordenados por la *quality* ya implementada, sin razonamiento por habitaciones. Mide si la calidad semántica del heatmap aporta sobre la proximidad pura (M2) y, junto con el sistema room-aware, permite atribuir la mejora al nivel correcto de la jerarquía.

Ambos baselines se implementan como variantes del pipeline existente y se comparan directamente con el sistema room-aware sobre las mismas métricas de la Sección 5.7.

6.2. Evaluación cuantitativa

Script automático que ejecuta N trials sobre objetos conocidos y produce:

- CSV con resultados por episodio (SR, SPL, Wrong Visits, CFR, CT2R, Rooms Before Success, etc.).
- Gráficas comparativas M1 vs. M2 vs. M3 vs. room-aware.
- Tablas para la memoria del TFG.

Para HSSD se utilizan las anotaciones nativas del `semantic_config.json` para generar trials con *ground truth* automatizado, sin etiquetado manual.

6.3. Objetos pequeños no representados en el VLMap

Usar `tools/place_small_objects.py` (sucesor de `place_objects.py`, con *deconfliction* de hosts y *grid offset* de 35 cm) para insertar objetos de interés que no aparecen en VLMap. Catálogo oficial tras la criba del 27.04:

- **Cocina (encimera):** `bottle`, `mug`, `teapot`, `toaster`, `coffee maker`.
- **Multi-room:** `laptop` (office/bedroom/living), `book` (bedroom/office/living).
- Colocados en posiciones conocidas para evaluación con *ground truth* a nivel de habitación.
- Visibles para YOLOE pero ausentes del VLMap pre-construido.

Quedan explícitamente fuera `kettle` (morfología confundible con `teapot`, mismo prior y mismas habitaciones; su presencia contaminaba la matriz de confusión sin medir nada nuevo) y `trash bin` (aparece en demasiadas habitaciones sin prior dominante, lo que vuelve indistinguible la búsqueda *room-aware* de una búsqueda plana y mide más al detector que al razonamiento).

Este conjunto de pruebas es el más representativo de la hipótesis del proyecto: para encontrar una taza, el robot tiene que razonar que vive típicamente en la cocina, sobre encimeras o mesas, y verificar visualmente con YOLOE. Un sistema sin los tres niveles falla por construcción en este escenario.

Próximo paso: N variantes del mapa por escena. La métrica actual evalúa una colocación canónica (objeto en su zona más probable) y mide si el robot la resuelve. La fase siguiente, ya prevista en el plan, consiste en generar N versiones del mismo mapa con los mismos objetos colocados en posiciones plausibles distintas (p.ej. botella en encimera, en mesa de comedor o en mesilla; libro en escritorio, mesilla o sofá). El robot ejecutará la misma query sobre las N variantes y se medirá la *degradación graceful* del sistema cuando el objeto no está en la habitación más probable. Es la prueba operativa de la hipótesis: un sistema room-aware debe degradar suavemente al moverse del prior al *long tail*, mientras que un sistema plano debería degradar abruptamente. La generación automática de variantes y la agregación cross-variant queda como tarea posterior a `post_placement_v2`.

7. Apartado extra: comparación baseline vs. executor

A fecha 22.04 existe ya una **implementación funcional** del *executor* y de un *entrypoint* alternativo, pero el apartado sigue **parcial**: la infraestructura de comparación ya está montada, pero falta ejecutarla a escala y consolidar resultados finales. Su objetivo no es añadir capacidades

nuevas, sino **poder comparar de forma controlada la política *inline* actual contra la nueva política basada en acciones tipadas**, una vez ambas están funcionando. Se aborda únicamente cuando todo el pipeline room-aware (Fases F y G) y las métricas (Fase H) estén operativos.

Motivación

`interactive_object_nav.py` acumula ≈ 2500 líneas de lógica *inline* validada contra el simulador. Reescribir ese fichero para introducir la nueva política sería un riesgo de regresión enorme y haría imposible comparar versiones. La estrategia es la opuesta: **no se toca el baseline**. Se crea una pista paralela que reutiliza las mismas primitivas de bajo nivel y solo cambia la orquestación.

Estructura propuesta

- `application/interactive_object_nav.py` — se queda como **baseline** estable. Mismo arranque, mismo *setup*, misma UI, mismo bucle *inline* actual.
- `application/interactive_object_nav_executor.py` — **entrypoint nuevo**. Reutiliza el arranque, el *setup* y la UI del baseline, pero su bucle principal pasa por `choose_next_action()` → `execute_action(...)`. No duplica primitivas. Su primera versión ya está implementada y expuesta en el menú interactivo del contenedor.
- `vlmaps/policy/executor.py` — **módulo nuevo**. Implementa `execute_action(robot, action, context, ...)` y despacha cada `Action` sobre las primitivas existentes: `go_to_room` → navegación a *room goal*; `explore_room` → *frontier picker* de Fase F; `inspect_candidate` → selección de candidato + *approach goal*; `verify_target` → YOLOE + scan 360° + centrado visual; `done` → cierre con *overlays*.
- Lo que ya funciona (parser de instrucción, heatmap, selección de candidato, navegación a *room goal*, *approach goal*, YOLOE, scan 360°, centrado visual, *overlays*) se **reutiliza tal cual**; el ejecutor solo lo invoca con otro orden y otra política de selección.
- Más adelante, el LLM se conecta únicamente como productor de `Action` JSON dentro de `choose_next_action()`, con la heurística de Fase D como *fallback*. Nada se vuelve monolítico.

Qué se compara

Tener los dos entrypoints conviviendo permite ahora un diseño de evaluación **2x2** con dos ejes independientes:

1. **Eje de orquestador**: baseline (`interactive_object_nav.py`) frente a executor (`interactive_object_nav_executor.py`).
2. **Eje de heatmap**: heatmap baseline (mapa crudo antes del postproceso) frente a heatmap postprocesado (`postprocess_heatmap`).

Combinando ambos ejes aparecen cuatro métodos comparables sobre el mismo JSONL:

Clave	Configuración
Ob_Hb	orquestador baseline + heatmap baseline
Oe_Hb	executor + heatmap baseline
Ob_Hp	orquestador baseline + heatmap postprocesado
Oe_Hp	executor + heatmap postprocesado

Criterio de cierre

El apartado se da por cerrado cuando: (i) el baseline sigue funcionando sin regresiones tras introducir el ejecutor; (ii) existe un informe lado a lado de los dos entrypoints sobre el mismo conjunto de *queries*; (iii) se publican las imágenes del heatmap baseline y del heatmap procesado para al menos una escena HSSD representativa.

A partir de ahí, la migración a ROS 1 / Toyota HSR (sección siguiente) puede acometerse sustituyendo *solo* las capas de percepción, navegación y ejecución, sin tocar la capa de decisión, que ya está validada en simulación.

Estado actual (22.04): las baterías 2x2 y de heatmaps están integradas; falta lanzarlas sobre escenas representativas y consolidar las métricas. La infraestructura queda dividida en dos pistas independientes que se pueden ejecutar por separado o en cadena:

- **Generador de *queries* compartido.** `tools/build_eval_queries.py` produce *batches* en formato JSONL bajo `tools/eval_queries/{scene}.jsonl` (50 queries por escena, 4 tipos: `object`, `room`, `room_object`, `compound`). Cada entrada lleva `id`, `target_label`, `expected_rooms` y `expected_room_polygons` (referencia exacta a polígonos del `semantic_config.json` para *ground truth* espacial). Ese mismo fichero alimenta los dos comparadores.
- **Runner online 2x2.** `tools/run_nav_eval.py` acepta ahora, además del *entrypoint*, un `heatmap_mode` (`baseline` o `postprocessed`) que se propaga como variable de entorno a los dos navegadores. Cada ejecución produce `raw_log.txt`, `setup.log`, un directorio `segments/` con un `.log` por query y un `manifest.json` enriquecido con `scene_id`, `scene_name`, `heatmap_mode` y toda la metadata del JSONL. Encima de esta pieza, `tools/run_full_2x2_eval.py` orquesta automáticamente las cuatro combinaciones `Ob_Hb`, `Oe_Hb`, `Ob_Hp` y `Oe_Hp`, copiando además el JSONL bajo `/workspace/results/eval_runs/{timestamp}/queries/`.
- **Agregador de métricas 2x2.** `tools/aggregate_full_2x2_eval.py` consume los cuatro manifiestos y produce dos vistas: (i) `compare_full.csv` y `compare_full.md` con una fila por query y columnas para las cuatro combinaciones, y (ii) `aggregate_metrics.csv` y `aggregate_metrics.md` con métricas agregadas por método, por `query_type` y por `tag`. Las métricas actualmente calculadas de forma robusta son: SR, Object SR, Room SR, CFR, CT2R, Rooms Before Success, Inspections Before Success, Wrong Visits, Mean Pose Updates, Early Stop Rate, Final Room Accuracy, Preview No Action Rate, Path Stopped Early Rate, Mean Executor Actions y Mean Room Transitions. Para ello, tanto baseline como executor emiten al final de cada query un marcador estructurado `[eval-summary] {...}` con historial de habitaciones, candidatos intentados/confirmados y rastro de acciones.
- **Comparador offline de heatmaps.** `tools/eval_heatmap_postprocess.py` sigue calculando, para cada query, el *heatmap* crudo (CLIP + proyección 2D + decay) y el *heatmap* postprocesado, pero ahora añade además agregación por `query_type` y por `tag` en `aggregate_by_slice.csv`. El `summary.md` incluye tanto la tabla global como tablas resumidas por tipo de query y por etiqueta (`multi_instance`, `ambiguous_room`, etc.), mientras que `per_query.csv` conserva la traza completa query-a-query.
- **Submenú de evaluación simplificado.** El menú interactivo del contenedor concentra ahora la evaluación bajo una única opción `t) Testing / evaluation submenu` con exactamente tres entradas visibles: `Generate test set`, `Compare full 2x2 pipeline` y `Heatmap-only offline analysis`. El usuario ya no tiene que encadenar scripts manualmente: basta elegir escena(s) y, si quiere, un JSONL alternativo; el *tooling* crea la estructura estándar de resultados bajo `/workspace/results/eval_runs/{timestamp}/`.

Como ambas pistas comparten `id`, `scene_id`, `query_type`, `expected_rooms` y `tags`, los resultados se pueden cruzar a posteriori para responder preguntas del tipo «¿las queries que el executor

falla son justo las que ya tenían un Hit@1 bajo en el heatmap raw, o hay fallos genuinos de orquestación?». El trabajo restante ya no es infraestructura, sino lanzar las baterías sobre escenas HSSD representativas, revisar salidas y comentar resultados.

Protocolo de evaluación finalmente adoptado

La batería 2×2 inicial se descartó por ser muy costosa de lanzar y porque mezclaba dos preguntas distintas. Se reemplazó por un **protocolo secuencial en dos etapas** que aísla cada pregunta y reduce el tiempo de cómputo manteniendo la potestad de responder ambas:

1. **Etapla 1 — selección del heatmap** (*offline, sin navegación*). Sobre 500 queries por escena se compara el heatmap raw (CLIP → proyección 2D → *decay*) con el heatmap **postprocessed** (`postprocess_heatmap()`). El criterio de victoria es agregado: *Hit@1*, *Hit@5*, *IoU top-mass-50*, masa proyectada dentro de la habitación esperada y proporción de masa fuera. La salida fija el modo a usar en la siguiente etapa.
2. **Etapla 2 — selección del orquestador** (*online, con Habitat-Sim*). Sobre 100 queries por escena, con el heatmap fijado al ganador de la Etapa 1, se compara el orquestador *baseline* (`Ob_Hp`) con el *executor* de acciones tipadas (`Oe_Hp`). Aquí las métricas son de navegación: SR, CFR, Wrong Visits, Pose Updates y Early Stop Rate.

Esta separación permite que la decisión sobre el heatmap se tome con un volumen mucho mayor de queries (500 vs. 100) y a coste cero de simulación, mientras que la simulación se reserva para el contraste de orquestador, que es lo que realmente exige ejecutarse en Habitat.

Etapla 1: heatmap raw vs. postprocessed

Lote de referencia: `results/heatmap_phase/20260424_153612/` (500 queries × 3 escenas HSSD). La Tabla 1 recoge los valores por escena y la Figura 2 muestra las cuatro métricas clave por escena.

Escena	n	Hit@1		Mass in expected		Wrong-room mass	
		raw	postp.	raw	postp.	raw	postp.
102344193_0	500	0.430	0.430	0.374	0.400	0.564	0.589
102344280_0	500	0.316	0.414	0.281	0.310	0.577	0.551
108736884_177263634_4	500	0.326	0.352	0.276	0.298	0.611	0.604

Cuadro 1: Etapla 1: comparativa offline del heatmap **raw** frente al **postprocessed** sobre 500 queries por escena. *Hit@1* mide si la celda de máxima masa cae dentro de la habitación esperada; *mass in expected* es la fracción de masa total proyectada dentro de las habitaciones correctas; *wrong-room mass* es la masa proyectada fuera (cuanto menor, mejor). Negrita = mejor entre los dos modos.

Análisis Etapla 1. El postprocesado mantiene o mejora *Hit@1* y *mass in expected* en las tres escenas, mejora *wrong-room mass* en dos de tres, y empata o cae ligeramente en *IoU top-mass-50* (el postproceso compacta los componentes y reduce la masa total, lo que penaliza una métrica de solapamiento de masa). En agregado el efecto neto es positivo, con la mejora más clara en la escena 102344280_0 (*Hit@1* 0.32 → 0.41, +9.8 puntos). **Decisión:** se fija `heatmap_mode = postprocessed` para la Etapla 2.

Etapla 2: orquestador Ob_Hp vs. Oe_Hp

Lote de referencia: `results/orchestrator_phase/20260424_154155/` (100 queries × 3 escenas, heatmap fijado a **postprocessed**). `Ob_Hp` es el orquestador *inline* actual; `Oe_Hp` es el nuevo

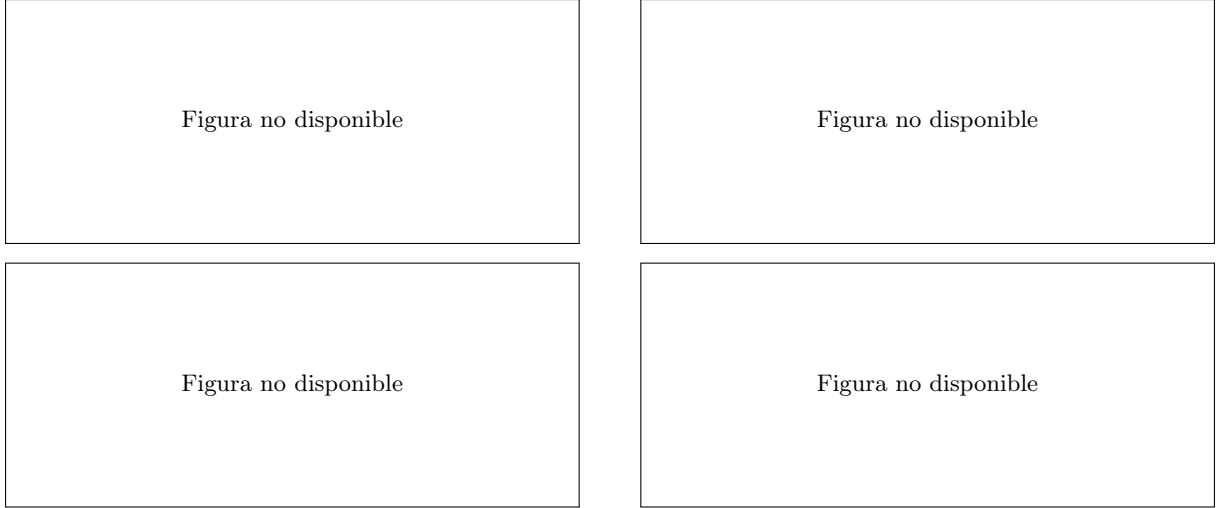


Figura 2: Etapa 1: calidad semántica del heatmap por escena. *Hit@1* (arriba-izq.), *mass in expected room* (arriba-der.), *IoU top-mass-50* (abajo-izq.) y *wrong-room mass* (abajo-der.). Cada barra compara el modo **raw** con el modo **postprocessed**; el postproceso gana en *Hit@1* y *mass in expected* en las tres escenas.

executor que pasa por `choose_next_action()` $\rightarrow$ `execute_action()` sobre las mismas primitivas. Las Tablas 2 y 3 resumen los resultados; las Figuras 3 y 4 muestran las métricas en formato gráfico por escena.

Método	SR	CFR	CT2R	Wrong Visits	Pose Updates
Ob_Hp (baseline)	0.667 $\pm$ 0.18	0.137 $\pm$ 0.07	0.290 $\pm$ 0.07	0.16 $\pm$ 0.12	382.4 $\pm$ 19.8
Oe_Hp (executor)	0.617 $\pm$ 0.14	0.177 $\pm$ 0.07	0.327 $\pm$ 0.07	1.18 $\pm$ 0.44	431.6 $\pm$ 35.9
Δ (Oe–Ob)	–0.050	+0.040	+0.037	+1.02	+49.1

Cuadro 2: Etapa 2: agregado cruzado de las 3 escenas (300 queries). SR = Success Rate, CFR = Completion Failure Rate, CT2R = Cost to Result. Negrita = mejor de los dos métodos. El baseline gana en las cinco columnas.

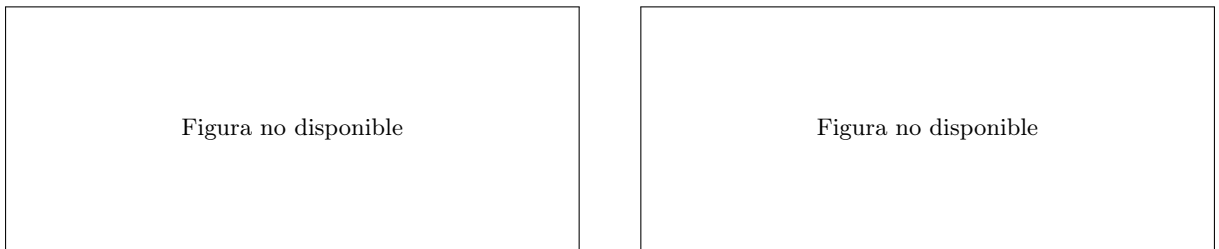


Figura 3: Etapa 2: Success Rate (izq.) y Completion Failure Rate (der.) por escena para Ob_Hp y Oe_Hp.

Análisis Etapa 2. El baseline Ob_Hp gana en SR cruzado (0.667 vs. 0.617, $\Delta = -5$ puntos) y en CFR (0.137 vs. 0.177). La diferencia más relevante no es el SR sino el coste: el executor genera $\sim 7\times$ más *Wrong Visits* (1.18 vs. 0.16) y un 13 % más de *Pose Updates* (432 vs. 382). Por escena, el baseline gana limpiamente en sc1 (0.92 vs. 0.81) y sc3 (0.58 vs. 0.54), y empatara en sc2 (0.50/0.50). Por tipo de query, el executor sí mejora ligeramente en *room_object* en sc2 (0.43 vs. 0.37), donde la decisión tipada explícita ayuda a aprovechar el *room hint*; en *object* puro pierde en las tres escenas. **Decisión:** el executor se mantiene en el repositorio para iterar sobre el *action*

Escena	Método	SR	CFR	Wrong Visits	Pose Updates	Early Stop
102344193_0	Ob_Hp	0.92	0.24	0.00	404.0	0.21
	Oe_Hp	0.81	0.27	0.56	381.2	0.04
102344280_0	Ob_Hp	0.50	0.09	0.28	387.2	0.02
	Oe_Hp	0.50	0.10	1.55	451.4	0.01
108736884_177263634_4	Ob_Hp	0.58	0.08	0.20	356.2	0.07
	Oe_Hp	0.54	0.16	1.43	462.2	0.03

Cuadro 3: Etapa 2 desglosada por escena. Negrita = mejor de los dos métodos en esa escena. El baseline gana o empata en SR en las tres escenas, y siempre gana en *Wrong Visits* y *Pose Updates*.

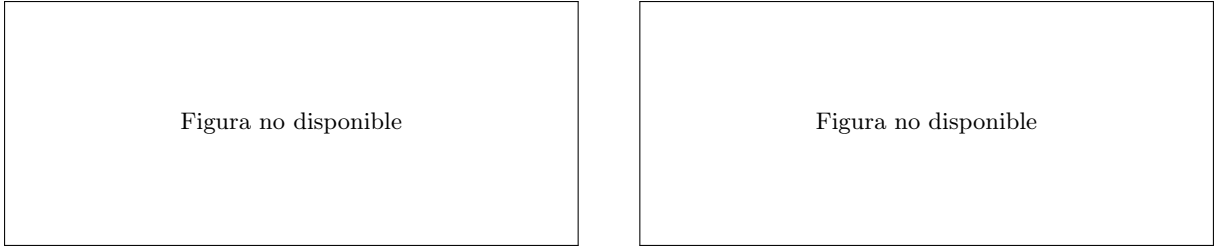


Figura 4: Etapa 2: *Wrong Visits* (izq.) y *Mean Pose Updates* (der.) por escena. La diferencia más visible no está en el éxito sino en el coste: el executor inspecciona muchos más candidatos incorrectos antes de cerrar la query.

vocabulary, pero el orquestador de referencia para el resto del proyecto sigue siendo Ob_Hp, con la salvedad de explorar el *action vocabulary* para tareas explícitamente *room-guided*.

Lo que estas dos etapas dejan asentado. (i) El postproceso del heatmap es la elección por defecto; (ii) el cuello de botella ya no es el modo de heatmap sino la calidad de la política de inspección dentro de la habitación; (iii) el executor sólo recupera al baseline en escenarios con *room hint* explícito, lo que indica que el camino para que valga la pena pasa por enriquecer el *action vocabulary* y conectarle el LLM estratégico (Fase G), no por reescribir más primitivas.

Calibración del umbral de confianza de YOLOE

El verificador visual YOLOE actúa como filtro final del pipeline: cuando el agente llega a un candidato propuesto por el heatmap, decide si la detección supera un umbral de confianza (τ) para confirmar el objetivo. Un τ demasiado bajo aceptaría detecciones débiles (riesgo de falsos positivos); uno demasiado alto rechazaría detecciones reales (falsos negativos), forzando al agente a seguir explorando.

Para fijar este parámetro de forma justificada se realizó un barrido controlado sobre $\tau \in \{0.30, 0.40, 0.50, 0.60\}$ en una única configuración del 2×2 (executor + heatmap postprocesado, Oe_Hp) y una única escena (102344193_0, 50 queries de muebles y categorías VLMMap). Como las decisiones de navegación, selección de habitación y ranking de candidatos no dependen de τ , basta con barrer un método para aislar el efecto del verificador; los otros tres métodos del 2×2 heredarán el valor seleccionado.

Análisis. La métrica CFR (*Correct First Room*) se mantiene constante en 0.18 para los cuatro umbrales, lo que confirma que el experimento aísla correctamente el efecto del verificador: la navegación y la selección de habitación no se ven alteradas. Las tres métricas que sí dependen del verificador (SR, Object SR y *found rate*, Figura 5) muestran su valor máximo en $\tau = 0.30$

τ	SR	Obj. SR	Found rate	Wrong visits	CFR
0.30	0.82	0.76	0.62	0.84	0.18
0.40	0.76	0.68	0.56	1.26	0.18
0.50	0.78	0.71	0.56	1.42	0.18
0.60	0.66	0.55	0.44	2.63	0.18

Cuadro 4: Métricas end-to-end para los cuatro umbrales evaluados sobre *Oe_Hp* en 102344193_0 (50 queries).

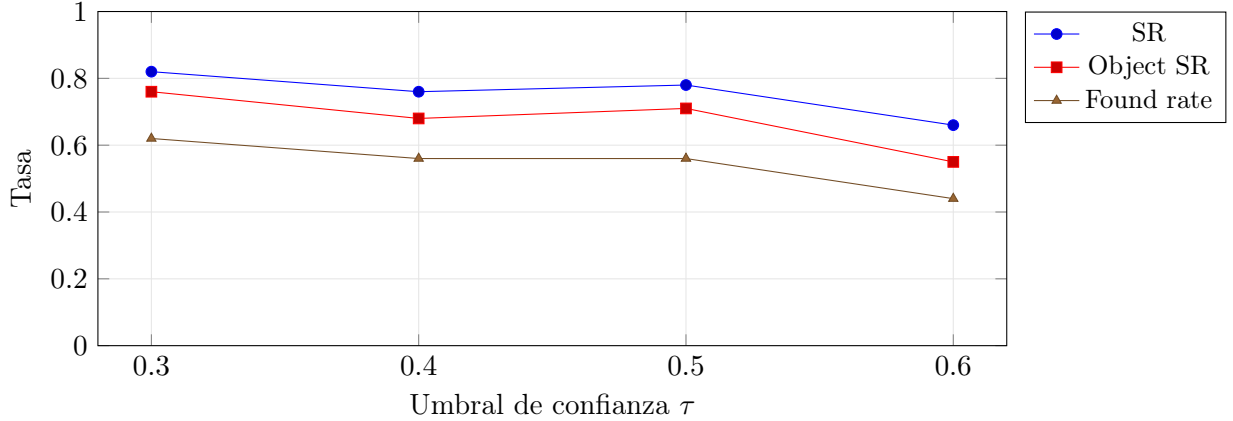


Figura 5: Evolución de SR, Object SR y tasa de detección al aumentar τ . La degradación es monótona a partir de $\tau = 0.40$.

y caen al aumentar el umbral. La tasa de visitas erróneas (Figura 6) crece de forma monótona, llegando a triplicarse entre $\tau = 0.30$ (0.84) y $\tau = 0.60$ (2.63): el agente inspecciona más muebles candidatos antes de aceptar uno como verificado.

Decisión. Se selecciona $\tau = 0.30$ por maximizar SR y Object SR y minimizar las visitas erróneas. Este valor se utiliza en todos los métodos del 2x2 oficial.

Limitaciones. La medida directa de la tasa de falsos positivos del verificador requeriría *queries negativas* (objetivos ausentes en la escena, donde cualquier `found=True` sería FP puro) o *ground-truth* a nivel de instancia para comparar la posición confirmada con la posición real del objetivo. Ninguno de los dos esquemas estaba disponible en este trabajo y se considera una línea de mejora del banco de evaluación.

8. Extensión futura: ROS 1 Noetic + Toyota HSR

El proyecto se desarrolla y valida primero en simulación, con HSSD como entorno principal y con foco en la capacidad de razonamiento por habitaciones y búsqueda semántica de objetos. Una vez consolidada la capa de decisión de alto nivel y evaluado el sistema sobre tareas representativas, se contempla como línea de extensión la migración a ROS 1 Noetic y su integración con el robot Toyota HSR.

La arquitectura de razonamiento por habitaciones (Sección 5) está diseñada para ser independiente de la plataforma de ejecución: las capas de decisión (selector de habitación, LLM estratégico, estado de búsqueda) no dependen de Habitat-Sim. La migración sustituiría únicamente las capas de percepción, navegación y ejecución por sus equivalentes en robot real:

- `vlmap_server` (nodo ROS): carga el VLMa y expone un servicio `/vlmap/query` para

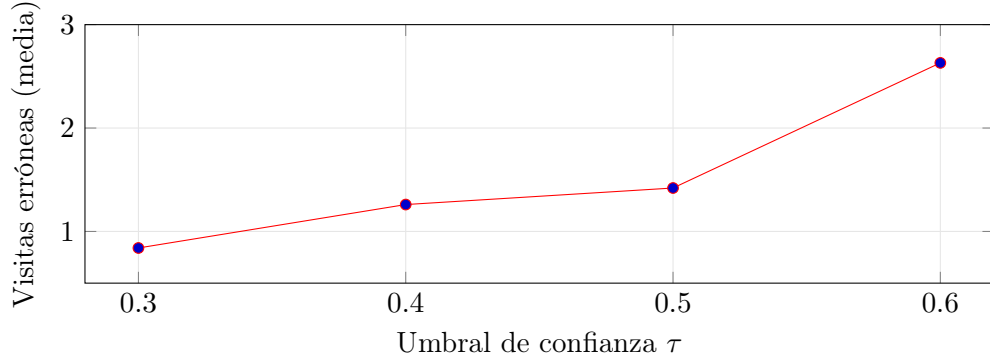


Figura 6: Visitas erróneas medias por query (candidatos inspeccionados pero no confirmados) en función de τ . El crecimiento monótono indica que el verificador rechaza cada vez más detecciones reales.

consultas semánticas.

- **vlmap_nav** (nodo ROS): recibe instrucciones en lenguaje natural, ejecuta el razonamiento por habitaciones y envía goals a **move_base**.
- Visualización en RViz: heatmap como **OccupancyGrid**, camino como **nav_msgs/Path**, estado de habitaciones como markers.
- Cámara RGBD del HSR como entrada para YOLOE (sustituyendo la cámara simulada de Habitat-Sim).

Esta extensión no forma parte del roadmap inmediato. Su viabilidad depende de que la capa de razonamiento funcione correctamente en simulación y de la disponibilidad del robot HSR en el laboratorio V4R del TU Wien.

Esquema futuro de doble contenedor

La evolución prevista del proyecto no consiste en “mover todo a ROS”, sino en mantener dos contenedores con responsabilidades nítidas. El contenedor **tfg-sim** seguirá siendo la referencia para HSSD/Habitat, datasets, evaluación y razonamiento semántico; **tfg-ros** absorberá ROS 1 Noetic, Gazebo, RViz y la interfaz hacia HSR-sim o el robot físico.

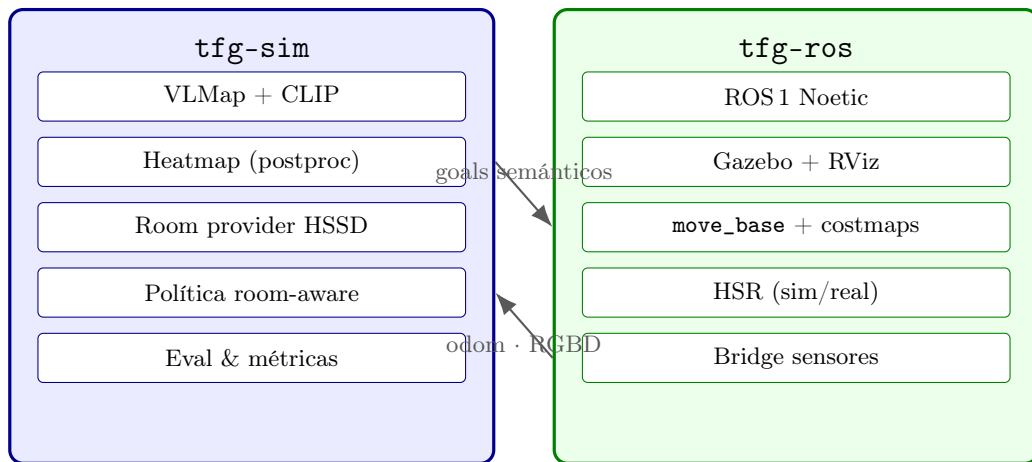


Figura 7: Arquitectura objetivo de doble contenedor. **tfg-sim** conserva la pila semántica completa; **tfg-ros** se encarga de la ejecución sobre HSR (simulado o físico). El acoplamiento se reduce a goals 2D y observaciones.

9. Diario de desarrollo

31 de marzo de 2026 — Correcciones del pipeline HSSD

1. **Mapa de obstáculos invertido:** modelo de dos bandas (suelo $h < 0,15$ m / obstáculo $0,2 \leq h < 1,5$ m). Una celda navegable = 1 si tiene suelo Y no tiene obstáculo.
2. **Heatmap ruidoso:** argmax binario reemplazado por score CLIP continuo con umbral 0.3 + max-pooling + difusión solo desde celdas con score alto.
3. **Robot atraviesa paredes en HSSD:** `sim.recompute_navmesh()` en `_setup_sim()` cuando `dataset_type == "hssd"`.
4. **Integración YOLOE:** `yoloe_utils.py` + `_yoloe_worker.py`, superposición sobre ventana 1st person.
5. **Camino visible:** `path_cells` pasado a `show_map()`, color rojo.
6. **Cámara primera persona:** inclinación $-\pi/8$ rad ($\approx 22,5$).

2 de abril de 2026 (primera iteración)

1. **Postproceso de heatmaps:** suavizado Gaussiano $\rightarrow$ umbral relativo $\rightarrow$ metadatos por componente (área, max, media, bbox, centroide).
2. **Exploración 360°:** pasos de 20°, detención al detectar.
3. **Ruta alternativa:** componente con mayor score a >20 celdas.
4. **Movimiento fluido:** sin `set_agent_state()` visible.

2 de abril de 2026 (segunda iteración)

1. **Worker YOLOE persistente:** modelo cargado una sola vez; protocolo stdin/stdout; inferencias ~ 100 –500 ms.
2. **YoloeSession:** modos síncrono y asíncrono (hilo bg).
3. **Scan 360° fluido:** pasos nativos de 5°, YOLOE en hilo bg.
4. **Cámara -30:** `habitat_utils.py`.
5. **Standoff 0,25 m:** acercamiento máximo al objeto.

13 de abril de 2026

1. **Bug protocolo stdout/stderr YOLOE:** `_ensure_mobileclip()` imprimía en stdout interceptando el "READY"; corregido a `sys.stderr.start()` ahora lee hasta recibir "READY".
2. **mobileclip_blt.ts:** búsqueda en múltiples rutas candidatas; eliminación automática de versión corrupta (<100 MB).
3. **Captura stderr del worker:** crashes visibles en consola.
4. **Dirección de cara definitiva:** $\arctan 2(-\Delta y, -\Delta x)$, signo consistente con el planificador.
5. **Sin teletransporte:** `plan_path_only()` + camino 800 ms antes de ejecutar.
6. **Hydra corregido:** `scene_dataset_config_file` sin prefijo + en ejecución y documentación de `menu.sh`.
7. **Repositorio GitHub limpio:** `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo.

8. **Bugs corregidos:** IndexError boundary (clamp en `plan_path_only()`) + umbral YOLOE subido a 0,40.
9. **Fórmula de calidad:** implementada en `postprocess_heatmap()`; componentes ordenados por quality.

14 de abril de 2026

1. **Reestructuración del documento de proyecto:** se redefine la finalidad del proyecto con foco en razonamiento por habitaciones.
2. **HSSD confirmado como dataset principal:** MP3D queda como soporte secundario; LabelMe diferido.
3. **Arquitectura room-aware diseñada:** 8 fases (A–H) con representación de estado por habitación, scoring de selección de room, acciones de alto nivel y rol estratégico del LLM.
4. **Roadmap corregido:** la escalera M1–M5 se reemplaza por la arquitectura room-aware como prioridad central. Baselines y evaluación pasan a fase posterior.
5. **Fase A implementada:** `SemanticSceneRoomProvider` construido con transformación completa `Habitat`→`grid` (`hab_to_grid`). Polígonos rasterizados en espacio grid. Cada componente del heatmap anotado con su habitación. Navegación a habitación por nombre funcional (centroide directo, sin heatmap ni YOLOE). Verificación visual de llegada a habitación omitida en simulación (coordenadas exactas); se contempla reactivarla para robot real con localización imperfecta.
6. **Parsing semántico por niveles:** identificado como trabajo futuro necesario. El parsing actual separa instrucciones compuestas en targets independientes; se requiere un parser que interprete habitación objetivo + objeto dentro de ella como una sola instrucción estructurada.

15 de abril de 2026

1. **Fase B completada:** `RoomState` y `SearchState` implementados en `vlmaps/utils/search_state.py`. Cada `RoomState` registra `explored_ratio`, `objects_seen`, `candidates_tried/confirmed`, `times_visited`, `target_relevance` y `target_found_here`. Se construye una instancia por target al inicio de cada instrucción y se actualiza incrementalmente durante la navegación y la verificación YOLOE. Al final de cada instrucción se imprime un resumen completo del estado de búsqueda.
2. **Fase C completada:** módulo `vlmaps/utils/room_priors.py`. Implementa `compute_room_priors(query, known_rooms, objects_seen_by_room)` fusionando tres fuentes con pesos $a = 0,5$, $b = 0,3$, $c = 0,2$: (a) LLM GPT-4o-mini con llamada estructurada al inicio de cada búsqueda de objeto; (b) tabla manual de ~ 35 objetos con scores por tipo de habitación; (c) evidencia de co-ocurrencia basada en `objects_seen` por habitación. El resultado normalizado se almacena en `RoomState.target_relevance` y se muestra como traza de depuración antes de cada consulta.
3. **Navegación solo a candidatos del heatmap:** eliminado el uso de `get_standoff_pos()` (que usaba `labeled_map_cropped`). Ahora el robot navega directamente al centroide del mejor componente de `kept_components` (filtrado real por `argmax + score > 0,3`). Esto evita que el robot viaje a zonas con señal VLMaap pero sin activación real en el heatmap.
4. **Filtrado de objetos navegables:** la lista de objetos mostrada antes de cada consulta usa el mismo filtro que `compute_heatmap` (vóxeles con `argmax ganador` y `score > 0,3`, mínimo 10 vóxeles), garantizando coherencia entre lo que se muestra y lo que el sistema puede

encontrar.

5. **Correcciones de navegación:** (a) `n_actions == 0` tratado como “ya en el destino” en vez de “sin camino”, corrigiendo el caso post-llegada a habitación; (b) eliminado el bloque de reintento por paso estrecho (*narrow-passagge retry*) que generaba más falsos positivos que beneficios; (c) umbral YOLOE subido a 0,3 y dilatación de obstáculos a 3 iteraciones (≈ 15 cm por lado).
6. **Tipos canónicos de habitación (*room_type* vs *room_instance*):** HSSD codifica instancias repetidas de un mismo tipo de habitación con sufijos numéricos (`bathroom.001`, `bathroom.002`, etc.). Se añade `canonical_room_type()` en `room_priors.py` que elimina el sufijo `.NNN` mediante regex (`re.sub(r'\.\d+$', '', s)`), y `compatible_room_types()` que devuelve el conjunto de tipos canónicos con `prior > 0,05`. Estas funciones permiten razonar sobre familias de habitación independientemente del índice de instancia.
7. **Comportamiento *local-first* al navegar (*room gate*):** nueva función `select_best_candidate(kept_c, current_room, query_priors, tried_centroids)` en `interactive_object_nav.py`. Si el robot ya se encuentra dentro de una instancia de habitación cuyo tipo canónico es compatible con el objeto buscado (e.g. robot en `bathroom.001`, objeto `sink`), el sistema prioriza los candidatos del heatmap situados en esa misma instancia, aplicando un bonus de +0,35 a su calidad efectiva. El cambio de habitación sólo se produce si (a) no quedan candidatos locales sin explorar, o (b) la calidad externa supera la local en más de un 40 % tras el bonus. El conjunto `_tried_centroids` en `SearchState` registra los centroides ya visitados para evitar ciclos.
8. **Navegación con margen de seguridad aumentado:** la dilatación de obstáculos en `habitat_lang_robot.py` se eleva a 5 iteraciones (≈ 25 cm por lado). La función `select_safe_goal_from_p` aumenta el radio mínimo de *clearance* de 3 a 5 celdas y adopta estrategia de máxima holgura: reúne *todos* los puntos del camino que cumplen el radio y selecciona el de mayor distancia al obstáculo más cercano (en vez del primero válido caminando hacia atrás). Si la pasada estricta no devuelve candidatos, una segunda pasada suaviza el umbral y vuelve a seleccionar por máxima holgura.
9. **Métricas de seguridad por camino:** nueva función `_compute_path_safety(path_cells, obs_map)` basada en `distance_transform_edt`. Registra longitud, holgura mínima y media, penalización acumulada por celdas con `clearance < 8` y coste seguro = longitud + $0,5 \cdot$ penalización. Los valores se imprimen en trazas `[safety]` tras cada planificación, facilitando el diagnóstico de rutas peligrosas y la futura migración a ROS/HSR.
10. **Corrección Bug 1 — comandos de habitación con validación real de llegada:** los comandos de tipo `kitchen`, `dining room`, etc. fallaban porque el sistema navegaba al centroide geométrico del polígono de la habitación (a menudo sobre un obstáculo), el planificador lo desplazaba a un nodo visgraph cercano (frecuentemente en otra habitación) y después declaraba éxito sin comprobar dónde estaba realmente el robot. Se implementa `find_reachable_room_goal()`, que enumera todas las celdas navegables del interior de la habitación objetivo, las ordena por holgura respecto a obstáculos y devuelve los top- K candidatos seguros. Tras la ejecución del camino se comprueba la habitación real con `get_room_at_cell()` y se valida mediante `_room_instance_matches()`, que acepta tanto coincidencia exacta como de tipo canónico (`kitchen.001` satisface el comando `kitchen`). Si la primera meta falla, se reintenta con los candidatos alternativos antes de declarar fracaso. Solo se imprime “Arrived at room X” cuando la comprobación pasa.
11. **Corrección Bug 2 — evidencia del heatmap domina en consultas directas:** para consultas de objeto con señal real en el heatmap (p.ej. `sofa`), los priors de habitación dependían excesivamente de los priors lingüísticos (LLM + tabla manual), ignorando que los

componentes del heatmap ya indicaban en qué habitación está el objeto. Además, nombres de habitación específicos de la escena como `tv` no coincidían con los fragmentos estándar (`living`) de la tabla manual. Se añade la tabla `_SEMANTIC_ALIASES` que mapea nombres atípicos a familias semánticas (`tv` $\rightarrow$ `living`, `laundryroom` $\rightarrow$ `laundry`, etc.) y se aplica en `_manual_prior()` y `_evidence_prior()` mediante `_normalize_room_for_priors()`. Se añade `compute_heatmap_room_evidence()` que agrega la calidad de componentes por instancia de habitación y normaliza a $[0,1]$. Se extiende `compute_room_priors()` con los parámetros opcionales `heatmap_evidence` y `query_type`: las consultas directas usan pesos $0,65 \cdot \text{heatmap} + 0,20 \cdot \text{manual} + 0,15 \cdot \text{LLM}$, mientras que las indirectas mantienen $0,45 \cdot \text{LLM} + 0,30 \cdot \text{manual} + 0,25 \cdot \text{evidencia escena}$. En el bucle de navegación, tras anotar los componentes con su habitación, se recalculan los priors con evidencia real antes de seleccionar el candidato, de modo que `sofa` con tres componentes en `tv` produce un prior $> 0,86$ para `tv`.

12. **Corrección Bug 3 — puerta de seguridad bloquea ejecución de rutas inseguras:** las métricas de seguridad [`safety`] existían pero eran puramente informativas: el sistema ejecutaba igualmente rutas con `goal_cl=0.0` o `min_cl=0.0`. Se añaden umbrales duros configurables `_MIN_GOAL_CLEARANCE=3,0` celdas y `_MIN_PATH_CLEARANCE=1,0` celdas. Antes de ejecutar cualquier camino a un objeto, se evalúan dichos umbrales: si el goal o la ruta no los superan, el centroide se marca como intentado en `_tried_centroids` y se prueban hasta tres candidatos alternativos. Si ningún candidato supera los umbrales, la consulta se omite con mensaje explícito. La seguridad ya no es anotación, sino control de flujo.
13. **Corrección Problema 1 — metas de aproximación seguras alrededor del centroide de componente:** el pipeline anterior planificaba hasta el centroide del heatmap (a menudo situado sobre un obstáculo), ejecutaba el camino completo y luego recorría el camino hacia atrás buscando un buen punto de observación. Si el centroide caía dentro de un obstáculo, el planificador visgraph producía rutas erróneas. Se añade `find_safe_candidate_approach_goals(robot_pos, safe_obs_map, robot_pos)` que muestrea posiciones de aproximación en un anillo de 8–25 celdas alrededor del centroide, filtra por navegabilidad en `_safe_obs_map` y holgura mínima ≥ 3 celdas, y ordena por holgura descendente y proximidad al robot. La meta de navegación se elige directamente del mejor candidato del anillo; el camino hacia atrás solo se activa como respaldo si el anillo queda vacío.
14. **Corrección Problema 2 — planificador global atravesaba paredes:** causa raíz doble. (a) La dilatación de obstáculos con 5 iteraciones (bilateral ≈ 10 celdas) cerraba los pasillos estrechos de HSSD (~ 10 celdas de ancho) desconectando habitaciones en el visgraph. Corregido volviendo a 3 iteraciones (≈ 6 celdas). (b) El método de recuperación de goal en espacio de obstáculo usaba `G.closest_point()` (holgura = 0) con `get_nearby_position()` como fallback, que devuelve `None` si las cuatro diagonales están bloqueadas; `vg.Point(None[0])` genera coordenadas inválidas $(-1, 96)$ y `G.shortest_path()` produce una línea recta a través de paredes. Ambas funciones se reemplazan por `_snap_to_nearest_free(p, obstacles, min_clearance)` en `navigation_utils.py`, que usa `distance_transform_edt` para encontrar la celda libre más cercana con holgura suficiente: el resultado está siempre garantizado dentro del espacio navegable del mapa del visgraph. Se añaden también registros de diagnóstico [`planner`] `Start navigable: ...` `Goal navigable: ...` y almacenamiento del mapa dilatado completo como `robot._safe_obs_map` para que el resto del sistema use la misma referencia que el visgraph. Las funciones `find_reachable_room_goal()` y `find_safe_candidate_approach_goals()` se actualizan para usar `robot._safe_obs_map` en lugar del mapa de navmesh crudo, eliminando la discordancia entre validación de metas y planificador.
15. **Corrección Problema 3 — escudo de colisiones local en ejecución:** `execute_nav_replay()` acepta ahora el parámetro opcional `dist_map` (transformada de distancias del mapa seguro

dilatado, precalculada una vez antes de cada ejecución). Antes de cada acción `move_forward`, el escudo predice la siguiente celda de cuadrícula a partir de la posición actual `curr_pos_on_map` y el ángulo `curr_ang_deg_on_map` del robot. Si la holgura en esa celda es inferior a `STOP_CL = 2,0` celdas, la ejecución se interrumpe inmediatamente y se activa la recuperación; si la holgura está entre `STOP_CL` y `SLOW_CL = 5,0` celdas se emite un aviso sin detener el movimiento. El escudo opera sobre el mismo mapa dilatado que el visgraph, de modo que «celda segura» tiene idéntico significado en planificación y en ejecución.

16. **Navegación one-shot — eliminados reintentos y recuperación:** el escudo de colisiones usaba el mapa dilatado (`_safe_obs_map`) para calcular la transformada de distancias, lo que hacía que el centro de un pasillo de ~ 10 celdas tuviera clearance ≈ 2 en el mapa dilatado y disparara paradas falsas al cruzar puertas. El escudo pasa a usar `robot.map.obstacles_map` (mapa crudo sin dilatar), que refleja el espacio físico real; los umbrales se ajustan a `STOP_CL = 1,0` y `SLOW_CL = 2,5` celdas. Adicionalmente, se elimina la llamada a `nav_recovery_and_replan()` tras un fallo de ejecución y el bucle de reintentos con metas alternativas en comandos de habitación. La navegación es ahora estrictamente one-shot: se planifica una vez, se ejecuta una vez y se reporta el estado real de la habitación; si la ejecución se interrumpe, el robot continúa desde la posición actual sin ruta nueva.

16 de abril de 2026

1. **Separación explícita entre inflación de planificación y escudo de ejecución:** el planificador global utiliza el mapa dilatado (`_safe_obs_map`, 3 iteraciones de dilatación binaria, ≈ 15 cm por lado) para trazar rutas con suficiente margen. El escudo de ejecución opera exclusivamente sobre la transformada de distancias del mapa crudo (`robot.map.obstacles_map`), sin dilatar. Esto garantiza que los pasillos estrechos de HSSD (~ 10 celdas de ancho, clearance real ≈ 5 celdas) no sean bloqueados falsamente por el escudo. La constante `_PLAN_DILATION_ITERS = 3` se declara de forma explícita en `habitat_lang_robot.py` y se imprime al inicio de la sesión junto con el mensaje: «*global planner only; execution shield uses raw map*».
2. **Modo de travesía de puertas (*doorway mode*):** antes de cada `move_forward`, el escudo mide las clearances perpendiculares a la derecha y a la izquierda del robot en la pose actual (`_side_l`, `_side_r`). Si ambas caen por debajo de `DOORWAY_SIDE_TH = 5,0` celdas y la clearance frontal supera `DOORWAY_FRONT_MIN = 0,5` celdas, se activa el modo de paso estrecho. En ese modo el umbral de parada dura se relaja a `STOP_CL_DOOR = 0,5` celdas (frente a 1,0 en espacio abierto) y cada paso se registra como «*cautious doorway traversal*». La transición ON/OFF se imprime explícitamente. El planificador puede enrutar libremente por puertas válidas; el escudo deja cruzarlas sin falsas paradas.
3. **Escudo sensible a la huella completa del robot:** el chequeo anterior solo evaluaba un punto (celda frontal prevista). Ahora se evalúan tres puntos de la huella en la pose predicha: *front-center*, *front-left* y *front-right* (desplazados lateralmente `ROBOT_HW = 2` celdas respecto al eje de avance). Se registra por separado `front clearance`, `left clearance`, `right clearance` y `footprint min clearance`. En espacio abierto, si algún lateral cae por debajo de `SIDE_STOP_CL = 1,5` celdas, el paso se bloquea. En modo puerta, el criterio de bloqueo es solo el mínimo de la huella frente a `DOORWAY_FRONT_MIN`.
4. **Puerta de entrada a habitación para confirmación YOLOE (*room-entry gate*):** hasta ahora el sistema podía declarar éxito inmediatamente si YOLOE detectaba el objeto desde la puerta antes de haber cruzado el umbral. Se añade una comprobación en los stages 0 y 1 de YOLOE: si el robot está en una habitación distinta de la que contiene el componente del heatmap objetivo (`best_comp["room"]`) y la distancia al centroide del candidato supera 20 celdas (≈ 1 m), la detección se marca como *tentativa* (`_yoloe_confirmed = False`) y la

búsqueda continúa con el stage siguiente. Solo se acepta como éxito definitivo si el robot ha entrado en la habitación correcta o se encuentra a menos de 20 celdas del candidato. Se registra en consola «*Object visible but room not yet entered → tentative only*» o «*Room entry confirmed*» según el caso.

5. **Centrado de trayectorias en pasos estrechos mediante eje medial local:** el visgraph seguía generando caminos geoméricamente válidos pero sesgados hacia uno de los lados del estrechamiento, lo que hacía que el escudo de ejecución bloquease pasos por clearance lateral insuficiente aun estando en modo de paso estrecho. El primer intento de corrección solo recentraba vértices ya existentes y no bastaba cuando el estrechamiento caía en mitad de un segmento largo del visgraph. Se añade un postprocesado `center_path_on_medial_axis()` en `vlmaps/utils/navigation_utils.py`. Tras obtener la ruta del visgraph, se rasteriza cada segmento y, si detecta una franja con clearance $< 4,0$ celdas, inserta un waypoint auxiliar en la zona más estrecha. Ese waypoint, junto con los intermedios originales, se desplaza hacia la celda de mayor holgura dentro de un disco local de radio 6, usando `distance_transform_edt` del mapa *sin* dilatar. El candidato solo se acepta si mantiene línea de visión libre con los waypoints vecino anterior y siguiente sobre el mapa de planificación dilatado, evitando atajos que corten esquinas o crucen paredes. Después se aplica un suavizado conservador de 3 puntos y se conserva la meta final sin desplazar. El resultado es que la ruta cruza por el centro del paso incluso cuando el cuello de botella no coincide con un vértice del visgraph, y el escudo vuelve a comportarse como mecanismo de seguridad, no como corrector sistemático de trayectorias.
6. **Modo de paso estrecho activado por geometría local, no solo por simetría lateral:** en ejecución real aparecía un caso límite al entrar oblicuamente en un marco o cuello de botella: un lateral del footprint caía por debajo de 1,5 celdas mientras el otro seguía aún en espacio relativamente abierto, de modo que el detector antiguo no activaba el modo de paso estrecho y el escudo bloqueaba el avance con la regla de espacio abierto. Se refuerza la detección en `execute_nav_replay()` con tres señales: (1) estrechamiento en la pose actual, (2) estrechamiento en la huella de la pose siguiente y (3) entrada asimétrica, cuando un lateral ya es crítico pero la clearance frontal sigue siendo cruzable. Además se añade histéresis de salida para mantener el modo activo mientras el robot permanezca dentro del cuello de botella. El modo pasa a modelar *pasos estrechos cruzables* en general, no solo puertas perfectamente simétricas entre habitaciones.
7. **Sustitución del *waypoint chasing* por un seguidor de polilínea con *lookahead*:** el aumento de waypoints auxiliares para centrar la trayectoria en pasos estrechos mejoraba la geometría, pero degradaba la ejecución con cientos de micro-correcciones de rumbo, porque el controlador discreto original perseguía cada subgoal por separado con cuantización fija de 5° y 0,1 m. Se densifica ahora la ruta planificada a nivel de celda mediante rasterización de segmentos y se ejecuta con un *path follower* sobre esa polilínea densa. En cada iteración, el robot proyecta su pose sobre la ruta, selecciona un objetivo adelantado (*lookahead*) varios pasos por delante y solo aplica la primera acción discreta necesaria para aproximarse a ese punto. El *lookahead* se reduce automáticamente dentro de pasos estrechos y se mantiene más largo en espacio abierto. Además, las métricas de seguridad y la selección de metas de observación pasan a evaluarse sobre la misma polilínea densificada, de modo que planificación, validación y ejecución comparten por fin la misma representación geométrica del camino.
8. ***Lookahead* recortado por visibilidad local del camino:** el primer seguidor con *lookahead* seguía teniendo un fallo importante en pasillos con giro o marcos oblicuos: podía elegir como objetivo un punto varios pasos por delante que ya estaba al otro lado de una esquina no visible desde la pose actual. Como el controlador discreto solo orienta hacia ese objetivo y aplica la primera acción, el robot intentaba cortar la esquina y acababa proponiendo

un avance con `front clearance = 0.0`. Se corrige haciendo que el *path follower* no use directamente el *lookahead* preferido, sino el punto más lejano de la polilínea densa que siga siendo alcanzable con línea de visión libre sobre el mapa de planificación dilatado. Así, dentro de cuellos de botella y giros cerrados, el seguidor deja de apuntar “a través de la pared” y se mantiene sobre el tramo realmente visible del camino.

9. **Etiquetas de llegada coherentes con el resultado real de la navegación:** en comandos de habitación se estaba mostrando “*Arrived: ...*” inmediatamente después de que terminara la ejecución del follower, incluso si el robot se había detenido antes del umbral y la comprobación final de habitación fallaba. Esto inducía a error durante la depuración visual. Se corrige la interfaz para que el overlay solo muestre “Arrived” cuando el robot ha alcanzado de verdad la habitación objetivo; en caso contrario, ahora se muestra “Not reached” en navegación a habitaciones o “Stopped before goal” en aproximaciones a objetos.
10. **La referencia de control vuelve a ser la ruta densa, no solo la polilínea compactada:** la versión que seguía únicamente la polilínea geométrica simplificada seguía permitiendo atajos laterales del controlador discreto. En la práctica, el robot podía “llegar más o menos” al final del segmento pero desviándose del eje real del camino, saliéndose hacia paredes o quedándose en la habitación vecina cuando el objetivo estaba cerca de una frontera semántica. Se reintroduce la trayectoria densa rasterizada como referencia de control, pero sin perseguir cada celda individualmente: el follower mantiene *lookahead* sobre esa ruta densa y la usa para medir progreso y error lateral. La polilínea compactada queda como representación del plan, mientras que la ejecución se ancla a la traza detallada que realmente debe seguir el robot.
11. **Banda muerta angular para eliminar micro-correcciones de 5°:** incluso después de volver a seguir la polilínea geométrica, el controlador discreto seguía tendiendo a consumir pequeños giros antes de casi cada avance cuando el error angular hacia el siguiente objetivo era muy bajo. Eso no cambiaba de forma significativa la seguridad, pero sí alargaba innecesariamente la ejecución en pasos estrechos. Se añade una tolerancia angular en el follower: si el objetivo inmediato ya está alineado dentro de un margen moderado, se prioriza `move_forward` en lugar de corregir rumbo con otro giro discreto. La tolerancia es más amplia en zonas estrechas que en espacio abierto. Con ello, se reduce el número de pasos y el patrón de micro-corrección sin introducir nuevos modos especiales de paso por puertas.
12. **El shield preventivo se elimina de la ejecución paso a paso:** el análisis fino mostró que el escudo local estaba mezclando dos nociones incompatibles: clearance sobre un mapa ya dilatado por el radio del agente y probes adicionales alrededor del cuerpo. Eso generaba falsos bloqueos en puertas y convertía la seguridad en el mecanismo principal de decisión, cuando debería ser solo una red de protección. La arquitectura actual elimina el *pre-step shield* y delega la colisión física en la navmesh de Habitat. La robustez pasa a depender de dos piezas más limpias: planificación *clearance-aware* y seguimiento fiel del camino.
13. **Watchdog de no-progreso real en el follower:** además del fallo semántico del shield, aparecía un segundo problema en ejecución: el robot podía quedarse vivo cientos de iteraciones repitiendo prácticamente la misma pose y agotando el *step budget*. Se añade un detector de no-progreso que vigila tanto la celda actual como el índice efectivo de progreso sobre la polilínea. Si durante varias iteraciones no cambia ni la celda útil ni el progreso útil, la ejecución se aborta pronto. También se detecta el caso en que el controlador no produce acciones válidas repetidamente para el mismo target local. La versión inicial de este watchdog abortaba incorrectamente durante la fase de giro in situ, porque tomaba “misma celda” como sinónimo de “sin progreso” incluso cuando la orientación seguía cambiando de forma útil hacia el siguiente tramo del camino. Se corrige para que una rotación real también cuente como progreso temporal del follower. Con ello, los fallos pasan a ser

rápidos y diagnosticables en lugar de consumir cientos de pasos inmóviles o abortar antes de empezar a avanzar.

14. **Reenganche explícito al camino cuando el robot se sale del eje:** incluso sin *shield*, el follower seguía pudiendo comportarse de forma errática si la banda muerta angular le dejaba avanzar cuando ya se había separado lateralmente del trazado. Se añade una lógica de *off-track reattachment*: la distancia del robot a la ruta densa se mide en cada iteración y, si supera un umbral, el *lookahead* se acorta automáticamente para volver a engancharse al eje del camino antes de permitir más avance agresivo. Además, la banda muerta angular solo puede forzar `move_forward` cuando el error lateral sigue siendo pequeño.
15. **Llegada más estricta en navegación a habitaciones:** la lógica anterior consideraba alcanzado el objetivo si el robot quedaba a pocas celdas del final del camino. Eso era suficiente para aproximaciones a objetos, pero no para comandos de habitación cerca de fronteras semánticas: el robot podía detenerse a 1–2 celdas del goal y seguir clasificado en la estancia vecina. Se introduce una tolerancia de llegada más estricta para navegación a habitaciones, de modo que el follower no dé por completado el camino hasta estar realmente encima del objetivo final o prácticamente superpuesto a él.
16. **Deadband angular menos restrictiva en el follower:** la primera versión de la banda muerta angular seguía siendo demasiado conservadora porque solo permitía sustituir un giro por un avance si el segundo elemento del plan local del controlador ya era `move_forward`. En la práctica, eso dejaba fuera muchos casos útiles en los que el error angular ya era pequeño pero el controlador aún proponía varios giros discretos consecutivos. Se elimina esa condición adicional: ahora el follower prioriza `move_forward` siempre que el error angular esté dentro de la tolerancia y la distancia al objetivo inmediato siga justificando el avance. Esto reduce el patrón “giro-giro-giro-avance” en puertas y pasillos angostos sin cambiar la arquitectura general del controlador.

20 de abril de 2026

1. **Validación en vivo dentro del contenedor de simulación:** en lugar de seguir depurando solo con logs pegados manualmente, se pasa a ejecutar pruebas reales dentro de `tfg-sim` sobre la escena HSSD usada durante el desarrollo (`scene_id=1`). Esto permitió comprobar de forma reproducible tres consultas representativas: navegación a habitación (`kitchen`), búsqueda de objeto con ambigüedad semántica local (`sofa`) y búsqueda de objeto con varios candidatos plausibles (`sink`). La prueba en vivo mostró que `kitchen` seguía fallando aunque el *shield* ya no bloqueaba puertas, mientras que `sofa` y `sink` sí llegaban a confirmarse con YOLOE. Con ello quedó claro que el problema residual principal ya no estaba en la planificación ni en la colisión preventiva, sino en el seguimiento local del camino.
2. **Corrección del bucle `preview produced no action`:** el fallo real de `kitchen` no era una colisión contra paredes, sino un colapso del follower cerca de un tramo concreto del camino, donde el objetivo local quedaba tan cerca que el controlador discreto devolvía repetidamente una previsualización vacía (`progress=58`, `target=59`). El follower seguía reenganchándose a ese mismo índice una y otra vez y acababa abortando sin salir del cuello. Se corrige en dos pasos. Primero, el objetivo visible de *lookahead* ya no puede ser arbitrariamente cercano salvo cuando el robot está realmente desviado o casi en la meta. Segundo, cuando un target local sigue produciendo una previsualización vacía, el sistema eleva explícitamente un *progress floor* y fuerza al follower a no volver a anclarse en ese mismo punto de la ruta. Con este cambio, la consulta `kitchen` deja de abortar en el mismo progreso y completa la trayectoria hasta quedar clasificada correctamente dentro de la cocina.

3. **La calidad semántica del heatmap deja de premiar blobs minúsculos:** el caso de `sofa` reveló otro problema de calidad. La fórmula anterior para puntuar componentes del heatmap aún permitía que un blob muy pequeño pero extremadamente denso compitiera con una región grande correspondiente al sofá real. En la práctica, el robot podía priorizar un sillón o una activación residual cercana solo porque el componente era compacto, aunque el soporte espacial del sofá fuese mucho mayor. Se reformula la *quality* de cada componente para favorecer de forma más explícita el soporte semántico real (`sum_val`) y el tamaño útil del componente, reduciendo el peso relativo de activaciones pequeñas. Tras este cambio, en la consulta `sofa` el componente grande asociado al sofá pasa a quedar por delante de los blobs minúsculos, evitando que el robot “vaya al sillón” salvo que semánticamente sea de verdad el mejor candidato.
4. **Overlay y visualización más fluidos sin cambiar de backend:** también se revisa la parte puramente visual del sistema. El overlay de navegación mostraba etiquetas del tipo `[64/366] -> sofa`, pero ese total era poco informativo para el usuario y además enfatizaba lo largo que era el seguimiento discreto. Se simplifica el texto en pantalla para mostrar solo el paso actual (`[64] -> sofa`). A nivel de fluidez, no se migra al visualizador 3D de Habitat, porque eso aumentaría la complejidad sin resolver el cuello principal. En su lugar, se mantiene OpenCV y se hace más eficiente: la vista en primera persona se sigue actualizando en cada acción, pero el mapa cenital se redibuja con una cadencia menor y el retardo explícito por paso se reduce prácticamente a cero. Así, la reproducción visual resulta mucho más fluida sin penalizar perceptiblemente el rendimiento de la simulación ni complicar la arquitectura de depuración actual.
5. **Fase D implementada en el pipeline real:** el razonamiento por habitaciones deja de ser una traza auxiliar y pasa a intervenir en la decisión. Se añade una función `select_best_room()` que, a partir de `SearchState`, la evidencia del heatmap y la posición actual del robot, calcula un `room_score` por habitación con cuatro señales ya disponibles en el sistema: prior semántico, evidencia agregada del heatmap, coste BFS sobre el mapa libre y penalización por intentos fallidos previos en esa habitación. La habitación ganadora se usa inmediatamente para filtrar `kept_components` antes del ranking fino de candidatos, de modo que el flujo deja de ser puramente global. La implementación actual utiliza una proxy práctica de *unexplored* basada en visitas y fallos acumulados, y deja el seguimiento de cobertura detallada para la futura Fase F.
6. **Ajustes del flujo de búsqueda visual y preparación de tests por lote:** tras estabilizar la Fase D se corrigen varios detalles del flujo objeto→detección. Primero, la selección de goals de aproximación deja de aceptar puntos que, aun teniendo clearance y perteneciendo a la misma habitación, quedarían geométricamente al otro lado de una pared respecto al centroid del candidato; ahora se exige además línea libre sobre el mapa de planificación. Segundo, las consultas directas a muebles presentes en la escena (por ejemplo `sofa` o `sink`) dejan de pasar por el *room selector*: en ese modo se fuerza una política *nearest-first* sobre los candidatos del heatmap y el razonamiento por habitaciones se reserva para consultas indirectas o ambiguas. Tercero, cuando la query sí activa selección de habitación, se añade un *room stage* explícito: el robot navega primero a un goal interior seguro de la sala elegida y solo después realiza el acercamiento al candidato concreto, evitando verificar desde el centro de una habitación equivocada. Cuarto, la confirmación YOLOE pasa a congelar tanto la vista RGB anotada como el punto objetivo sobre el mapa cenital, dejando una marca persistente de la detección positiva. Finalmente, se preparan scripts de *batch testing* para generar automáticamente una batería de consultas con todas las habitaciones suficientemente navegables y con todos los objetos presentes en la escena, dejando el runtime real de esas pruebas para la siguiente ronda de validación manual.

21 de abril de 2026

1. **Fase F consolidada y accesible para validación manual:** se da por cerrada la capa de acciones de alto nivel introducida el día anterior y se revisa explícitamente su alcance real. La implementación no modifica todavía el comportamiento *runtime* del robot dentro de `interactive_object_nav.py`; su función es preparar la transición desde una política *inline* a una política explícita y tipada. Quedan definidos cinco tipos de acción (`go_to_room`, `explore_room`, `inspect_candidate`, `verify_target` y `done`), su validación estructural, su serialización JSON para la futura capa LLM y un selector de *frontier* intra-habitación para `explore_room`. En paralelo, `SearchState` amplía su papel como memoria del episodio con un registro ordenado de acciones y un rastro de celdas visitadas, que servirán como base del ejecutor de la siguiente fase.
2. **Pruebas unitarias y acceso desde el menú del contenedor:** para que la validación de Fase F no dependa de Habitat, de la GPU ni de una escena concreta, se mantiene una suite `pytest` autocontenida (`tests/test_phase_f_actions.py`) y, además, se expone su ejecución desde el menú interactivo del contenedor mediante una nueva opción `p`. Esta entrada lanza la batería de pruebas de la capa de política y muestra de forma inmediata si la representación de acciones, el *round-trip* JSON, el registro extendido de `SearchState` y el *frontier picker* siguen siendo consistentes. El objetivo aquí no es medir navegación real, sino garantizar que la interfaz de alto nivel sobre la que se montará la siguiente fase permanece estable antes de conectarla al pipeline operativo.
3. **Corrección del lanzador de *batch testing*:** al ejecutar la opción `t` del menú con dataset HSSD, el script `tools/nav_batch_queries.py` fallaba con `TypeError: string indices must be integers` en `HabitatLanguageRobot.__init__`. La causa era que el script cargaba la configuración con `OmegaConf.load`, que no resuelve la lista `defaults` de Hydra: el campo `data_paths` quedaba como la cadena literal `"default"` en lugar del diccionario con `habitat_scene_dir/vlmaps_data_dir`. Se sustituye la carga manual por `hydra.compose` con `initialize_config_dir`, pasando los *overrides* (`scene_id`, `dataset_type`, `scene_dataset_config_file`, `data_paths`) por la vía oficial de Hydra. Tras el arreglo, el script genera `queries.txt` correctamente y el batch llega a lanzar `interactive_object_nav.py`. Queda registrado además que un `scene_id` fuera de rango (p. ej. teclear 11 cuando solo existen 0 y 1) provoca un `IndexError` en `vlmaps_data_save_dirs[scene_id]` es una limitación de UX del menú, no un bug del pipeline, y se considerará añadir una validación previa.
4. **Primer corte funcional del ejecutor alternativo:** se materializa por fin el apartado “baseline vs. ejecutor” con una implementación inicial real. Se mantiene `interactive_object_nav.py` como *baseline* intacto y, en paralelo, se añade `application/interactive_object_nav_executor.py` como nuevo entrypoint junto con `vlmaps/policy/executor.py`. El nuevo módulo define un `ExecutorContext` y un despachador `execute_action(...)` que traduce las acciones tipadas de Fase F (`go_to_room`, `explore_room`, `inspect_candidate`, `verify_target`, `done`) a comportamiento real reutilizando las primitivas ya validadas del baseline: *room staging*, generación de *approach goals*, *path replay follower*, chequeos YOLOE en llegada y tras giro, *scan 360°* y centrado visual. La política de alto nivel se mantiene coherente con el estado del proyecto: si la query es una habitación, se genera `go_to_room` → `done`; si es un mueble directo presente en la escena, se mantiene *nearest-first* sin selector de habitación; y para consultas ambiguas o indirectas se conserva el flujo jerárquico “habitación primero, candidato después”. Además, el menú del contenedor incorpora una nueva opción `e`) **Interactive executor navigation**, que permite lanzar esta variante sin tocar comandos manuales. La parte *pendiente* ya no es construir el ejecutor, sino compararlo sistemáticamente frente al baseline con las métricas de Fase H y con volcados lado a lado del heatmap baseline y del heatmap procesado *room-aware*.

5. **Resolución explícita de instancias de habitación en el parser y en la comprobación de llegada:** se corrige un fallo funcional relevante del flujo de navegación por habitaciones. Hasta este punto, órdenes como `bathroom.001` o expresiones más naturales como `bathroom 1` terminaban colapsándose sobre la habitación base `bathroom`, porque el pipeline solo distinguía familias semánticas y no instancias concretas. Se introduce un resolvedor explícito de nombres de habitación en `room_provider.py`, capaz de mapear alias amigables (`bathroom 1`, `bathroom one`, `bedroom 2`) a sus etiquetas exactas de escena (`bathroom.001`, `bedroom.002`, etc.), manteniendo a la vez el comportamiento esperado para comandos base como `bathroom` o `bedroom`, que siguen prefiriendo la habitación principal cuando existe. Esta resolución se conecta tanto al *baseline* como al nuevo *executor*: tras el parse LLM se preservan menciones explícitas de instancia presentes en la instrucción original, la selección de *room goals* pasa a usar la instancia resuelta y la condición de llegada deja de aceptar que `bathroom` satisfaga `bathroom.001`. Se añade además una batería unitaria nueva (`tests/test_room_instance_resolution.py`) que valida la distinción entre habitación base e instancia numerada, junto con la compatibilidad con el resto de la capa de acciones de Fase F.
6. **Batería de comparación baseline vs. executor disponible desde el menú:** el apartado extra deja de ser solo una propuesta en papel y pasa a tener una infraestructura operativa para lanzar comparaciones controladas. Se añade al menú del contenedor la opción `v) Compare -- baseline vs executor`, que genera un lote común de *queries*, ejecuta ambas variantes del navegador sobre ese mismo fichero de entrada y resume el resultado en CSV y markdown. La implementación queda repartida entre `tools/nav_batch_queries.py` (generación del lote), `tools/run_hssd_nav_compare.sh` (orquestración host-side) y `tools/compare_nav_runs.py` (resumen query-a-query de detecciones, habitación final, abortos y número de actualizaciones de pose). No sustituye todavía a la evaluación de Fase H, pero elimina el trabajo manual repetitivo de abrir y comparar logs a mano.
7. **Primer corte funcional de Fase G sobre el executor:** se añade `vlmaps/policy/strategic_policy.py` como capa de decisión estratégica sobre estado estructurado. Esta pieza construye un *snapshot* del episodio (prior de habitaciones, evidencia del heatmap, shortlist de candidatos, frontiers sugeridos y últimas acciones), consulta opcionalmente a un LLM para pedir la siguiente **Action** en formato JSON y valida esa propuesta antes de ejecutarla. Si el LLM no está disponible o devuelve una acción inconsistente, el sistema cae automáticamente a una política heurística determinista, lo que evita romper el flujo ya validado del executor. A nivel de integración, `interactive_object_nav_executor.py` deja de preconstruir un plan fijo y pasa a operar como bucle “elegir siguiente acción → ejecutar acción”, con modo de política seleccionable desde el menú (`heuristic`, `hybrid`, `llm`). La validación unitaria queda cubierta por `tests/test_phase_g_strategic_policy.py`.

22 de abril de 2026

1. **El *testing* deja de ser una colección de utilidades y pasa a ser un flujo único:** se reorganiza por completo el submenú `t` del contenedor para que muestre solo tres opciones visibles: `Generate test set`, `Compare full 2x2 pipeline` y `Heatmap-only offline analysis`. Las antiguas utilidades intermedias (batch baseline aislado, comparación binaria baseline vs. executor, runner JSONL suelto y tests unitarios de F/G) siguen existiendo como piezas internas o de desarrollo, pero dejan de ser el flujo principal del usuario. La intención es que la evaluación pase a ser reproducible y autoexplicativa: elegir escena(s), pulsar, esperar y recoger resultados en una carpeta estándar.
2. **Selección explícita del tipo de heatmap en runtime:** hasta esta fecha, *baseline* y *executor* compartían implícitamente `compute_heatmap()`, lo que impedía montar una com-

paración 2x2 limpia sin duplicar entypoints. Se introduce `VLMAPS_HEATMAP_MODE` con dos modos explícitos: `baseline` (heatmap crudo tras proyección 2D + decay, sin postproceso) y `postprocessed` (pipeline actual con `postprocess_heatmap`). El `baseline` y el executor imprimen el modo activo al arrancar y el valor queda recogido en el `manifest.json` de cada corrida. Además, ambos entypoints emiten ahora al final de cada query un resumen estructurado `[eval-summary] {...}` con habitación final, historial de habitaciones, número de candidatos inspeccionados/confirmados, rastro de acciones y recuento de celdas visitadas. Esta capa permite calcular métricas de Fase H sin depender de *regex* frágiles sobre logs textuales.

3. **Runner online 2x2 y agregador de métricas completos:** se añaden dos herramientas nuevas. La primera, `tools/run_full_2x2_eval.py`, coordina de forma automática las cuatro combinaciones `Ob_Hb`, `Oe_Hb`, `Ob_Hp` y `Oe_Hp`, reutilizando `tools/run_nav_eval.py` como runner por método. La segunda, `tools/aggregate_full_2x2_eval.py`, consume los cuatro manifiestos y genera dos vistas listas para inspección: `compare_full.csv/.md` (una fila por query) y `aggregate_metrics.csv/.md` (una fila por método y *slice*). Las salidas se agrupan bajo una estructura homogénea: `/workspace/results/eval_runs/{timestamp}/queries/, pipeline_2x2/{scene}/Ob_Hb, ...`, junto con un `config.json` raíz que describe escenas, semilla, política del executor, rutas de queries y métodos corridos.
4. **Métricas online ya instrumentadas y *slicing* por query_type/tag:** el agregador 2x2 calcula ya de forma reproducible SR, Object SR, Room SR, CFR, CT2R, Rooms Before Success, Inspections Before Success, Wrong Visits, Mean Pose Updates, Early Stop Rate, Final Room Accuracy, Preview No Action Rate, Path Stopped Early Rate, Mean Executor Actions y Mean Room Transitions. No todas las métricas tienen aún el mismo grado de robustez: por ejemplo, `Inspections Before Success` se aproxima como `candidates_tried - candidates_confirmed` cuando la query termina en éxito, y `Rooms Before Success` se interpreta como el índice de la primera habitación esperada dentro de `visit_history`. Estas aproximaciones quedan explícitas en el `Markdown` generado, en vez de presentarlas como magnitudes más precisas de lo que el log realmente permite.
5. **La evaluación offline de heatmaps gana agregación por slices y envoltorio estándar:** `tools/eval_heatmap_postprocess.py` mantiene la salida `per_query.csv`, pero añade un `aggregate_by_slice.csv` y tablas adicionales en `summary.md` para resumir el rendimiento del heatmap raw y del heatmap postprocesado por `query_type` y por `tag`. Para que esta tercera opción del submenú no requiera rutas ni carpetas manuales, se añade `tools/run_heatmap_offline_eval.py`, que normaliza la salida bajo `/workspace/results/eval_runs/{t` y escribe también su propio `config.json`.
6. **Validación rápida sin navegación larga:** se ejecutan únicamente comprobaciones ligeras, coherentes con el criterio del proyecto de no lanzar desde el asistente corridas largas de simulación sin supervisión del usuario. Han pasado `bash -n` sobre el menú, `py_compile` de los scripts y entypoints tocados, una batería `pytest` nueva con manifests sintéticos para el agregador 2x2 (`tests/test_eval_2x2_tools.py`) y, además, las suites ya existentes de Fase F, Fase G y resolución de instancias de habitación (26 tests). Con ello queda validado el armazón de evaluación; lo que falta a partir de ahora es usarlo sobre escenas y JSONLs reales para producir resultados y discutirlos en la memoria.

23 de abril de 2026

1. **Limpieza del dataset HSSD:** se reduce el listado de escenas disponibles a las tres que se usarán en la evaluación oficial: `102344193_0`, `102344280_0` y `108736884_177263634_4`. Las restantes (variantes y dos carpetas con nombres corruptos generadas por capturas accidentales de `stderr` durante el *collect*) se eliminan de `data/vlmaps_dataset_hssd/`. El

menú interactivo refleja ya sólo `scene_id` 0/1/2.

2. **Reversión de la rama room-aware en el harness de evaluación:** tras un día con seis métodos (orquestador $\times$ heatmap $\times$ room-aware) se decide volver al 2×2 original (Ob_Hb, Ob_Hp, Oe_Hb, Oe_Hp). Razón: el switch room-aware no es una variable independiente de evaluación sino una decisión arquitectónica del executor, y mantenerlo como eje del harness multiplicaba el coste sin aportar comparaciones nuevas para el nivel actual (muebles y habitaciones). `tools/eval_methods.py` y `tools/run_full_eval.py` vuelven a la matriz de cuatro celdas.
3. **Calibración del umbral de confianza de YOLOE:** se introduce un sweep dedicado al verificador visual sin barrer los cuatro métodos del 2×2 . Justificación: el `conf-thresh` sólo afecta la fase de verificación al llegar al candidato; las decisiones de navegación, heatmap y selección de habitación son independientes. Por tanto basta con barrer un único método (Oe_Hp) para aislar el efecto del umbral.
4. **Nueva herramienta `tools/run_yoloe_thresh_sweep.py`** (encargada a Codex). Corre `run_nav_eval.py` repetidamente sobre la misma escena y JSONL, una vez por `threshold`, propagando el valor vía `VLMAPS_YOLOE_CONF_THRESH`. Al terminar, agrega los manifests y produce `compare_thresh.csv` y `compare_thresh.md` con una fila por `threshold` y columnas SR, Object SR, Found rate, Wrong visits, CFR, CT2R, Early Stop Rate. Convive sin tocar el harness 2×2 ni el submódulo `vlmaps`.
5. **Resultado del barrido sobre $\{0.30, 0.40, 0.50, 0.60\}$** (escena 102344193_0, 50 queries, método Oe_Hp): $\tau = 0.30$ maximiza SR (**0.82**), Object SR (**0.76**) y *found rate* (**0.62**) y minimiza las visitas erróneas (**0.84**). A partir de $\tau = 0.40$ la degradación es monótona: las visitas erróneas se triplican y la SR cae 16 puntos en $\tau = 0.60$. La constancia de CFR (0.18 en los cuatro runs) confirma que el experimento aísla correctamente el verificador y no contamina la navegación. Los detalles cuantitativos y la decisión están integrados en la subsección *Calibración del umbral de confianza de YOLOE* del bloque de testing.
6. **Discusión honesta sobre la medición de falsos positivos:** durante el análisis se descarta el indicador *FP-proxy* inicial (`found  $\wedge$   $\neg$ success` sobre queries `room_object`). Razones: (i) el *ground truth* del *benchmark* es a nivel de habitación, no de instancia, por lo que encontrar una instancia válida del mismo objeto en otra habitación se contabiliza erróneamente como falso positivo; (ii) el agente detiene la búsqueda en la primera confirmación, ocultando posibles FPs latentes en otras habitaciones; (iii) con sólo 8 queries `room_object` el indicador es estadísticamente inservible. Se documenta como limitación del banco de evaluación que una medición directa de FPs requeriría *queries negativas* (objetivo ausente en la escena) o *ground-truth* de instancia.
7. **Decisión adoptada:** se mantiene $\tau = 0.30$ como umbral de confianza para todos los métodos del 2×2 oficial. La elección queda justificada documentalmente (tabla, gráficos y discusión) en la sección de testing, evitando aparecer como un default arbitrario.
8. **Decisión estratégica: activar la migración a ROS 1 Noetic antes que el nivel 3.** Tras revisar el coste real de añadir nivel 3 (objetos pequeños) directamente sobre Habitat y luego portarlo a ROS, se concluye que esa secuencia obliga a iterar dos veces sobre los mismos componentes (Phase G, surrogate furniture, YOLOE bridge, harness). Se reordena el roadmap para migrar primero el stack a ROS 1 Noetic + Gazebo y desarrollar el nivel 3 directamente sobre el stack final. Plazo asumido para la migración completa: **un mes** (hasta el 23 de mayo de 2026).
9. **Renuncia explícita a HSSD como dependencia obligatoria:** las escenas pasarán a ser *house worlds* de Gazebo (HSR oficial, AWS RoboMaker) o iGibson (15 apartamentos basados en escaneos reales, utilizado por MoMa-LLM). El razonamiento por habitaciones,

surrogate furniture y verificación con YOLOE son agnósticos a la fuente de la escena.

10. **Esquema de etiquetado de habitaciones por centroides + Voronoi** (estilo MoMa-LLM): se sustituye el sistema de polígonos `semantic_config.json` por una partición de Voronoi sobre centroides marcados manualmente (un *click* por habitación). El mismo esquema vale para escenas Gazebo, iGibson y, más adelante, para el laboratorio físico. Coste de anotación: 5–10 minutos por escena. Implementación en `room_provider` estimada en una sesión manteniendo la interfaz pública.
11. **Inspiración explícita en MoMa-LLM** (Honerkamp et al., RSS SemRob 2024): adopción conceptual de tres ideas del paper, con re-implementación propia y *sin copiar código* (compromiso de integridad académica). Las ideas adoptadas son: (i) clustering de habitaciones por Voronoi; (ii) vocabulario de acciones de alto nivel (ya parcialmente cubierto por la Phase F existente); (iii) métrica AUC-E (área bajo la curva de eficiencia) como complemento al SR a *budget* fijo, para reportar resultados en la memoria.
12. **Continuidad de Docker en host distinto**: la migración mantiene Docker como mecanismo de empaquetado. El contenedor pasa de `tfg-sim` (Habitat-Sim sobre Ubuntu 20.04) a un nuevo contenedor Noetic+Gazebo sobre Ubuntu 20.04. El usuario sigue trabajando desde su host Ubuntu 22.04 sin reinstalar nada en el host. Cuando se traslade el proyecto a un equipo nativo Ubuntu 20.04, el contenedor sigue siendo válido (ROS nativo en el host se considera optimización opcional, no requisito).

24 de abril de 2026 — Inserción real de objetos pequeños y *deconfliction* del placer

1. **Reducción del dataset HSSD a tres escenas operativas**: tras la limpieza del 23.04, las escenas activas para evaluación de objetos pequeños quedan fijadas en 102344193_0 (single-floor, 8 habitaciones, balcony interior), 102344280_0 (15 habitaciones, ~447 m², escena principal del proyecto) y 108736884_177263634_4 (escena con *office* explícita, necesaria para validar el caso multi-room). Otras escenas evaluadas (107734254, 107734479 y variantes) se descartan: bancos y otomanas bloqueando el piano impedían el plan de movimiento del robot, y la cobertura del navmesh era irregular tras los *collect*. Las carpetas se eliminan de `data/vlmaps_dataset_hssd/` desde dentro del contenedor para evitar conflictos de permisos.
2. **Spawn fijo por escena en `collect_hssd_dataset.py`**: se añade el flag `--start-pos X Z` al script de colección. El valor se *snappea* al navmesh con `sim.pathfinder.snap_point(...)` y la Y se deja libre para que la lleve la malla. El menú interactivo (`docker/menu.sh`) define un diccionario `_SPAWN_POINTS` por `scene_id` y propaga el flag automáticamente, evitando *spawns* aleatorios encima de camas o dentro de armarios que invalidaban la colección.
3. **Inventario de objetos a colocar**: se redacta una primera *spec* JSON por escena bajo `tools/eval_queries/specs/small_objects_*.json` con ocho entradas (*bottle*, *laptop*, *mug*, *kettle*, *toaster*, *coffeemaker*, *teapot*, *trashbin*). Cada entrada lleva *furniture*, *room_hint* y *count*. Se crea `tools/place_small_objects.py` que carga la *spec*, busca instancias de la *furniture* objetivo dentro de la habitación indicada y escribe un nuevo `*.scene_instance.json` con los *placements* materializados. El fichero original se preserva como `*.before_placements.json`.
4. **Ampliación de `room_priors.py` y `object_priors.py` con los nuevos targets nivel-3**: se añaden entradas para *kettle*, *toaster*, *coffee maker*, *mug*, *trash bin*, *printer*, *basketball*, etc., tanto en la `_MANUAL_TABLE` como en la tabla de co-ocurrencia y en los `_SEMANTIC_ALIASES`. El cambio es aditivo: ningún prior existente se modifica.
5. **Diagnóstico del bug de placement (*kettle* 0/6)**: el primer *full eval* con la *spec* ini-

cial (50 queries) produce SR global 14% (*baseline*). Tras varias rondas de afinado de la pipeline de búsqueda (`post_fix_v1` → `v7`) se llega a SR 44% pero `kettle` sigue dando 0/6 en todos los runs. Inspeccionando el `scene_instance.json` resultante se descubre que *los seis targets de cocina* (`bottle`, `kettle`, `toaster`, `coffeemaker`, `mug`, `teapot`) comparten *exactamente las mismas coordenadas XYZ* sobre el primer `counter` del `kitchen`: el selector determinista del placer devolvía siempre el primer candidato y ninguna variabilidad espacial. Habitat colapsa los seis objetos en el mismo punto y YOLOE solo ve uno.

6. **Fix: *deconfliction* por (`template_id`, `XZ`) con *grid offset*:** el placer se modifica para mantener un registro `used_host_keys` y un contador `host_reuse_count`. Antes de aceptar un candidato, se reordena la lista de *candidates* de manera *fresh-first*: hosts no usados se prueban antes de reciclar uno ya ocupado. Cuando la reutilización es inevitable, se aplica un *offset* determinista en una rejilla 3×3 (`_grid_offset`) con paso de **35 cm** sobre *furniture* y de **30 cm** sobre suelo. El cambio es puramente aditivo y no toca la lógica de búsqueda. La verificación a mano sobre 102344193_0 muestra los seis objetos repartidos por el counter en la rejilla esperada.
7. **Resultado del *run* `post_placement_full`:** SR global pasa de 44% a **68%** (34/50). El salto de +24 puntos viene de un único cambio no relacionado con la búsqueda. `kettle` pasa de 0/6 a 3/6 y `teapot` de 4/8 a 7/8. Las regresiones (`trash bin` 3/6 → 2/6) se atribuyen al *floor offset* de 80 cm que mueve el bin fuera del centroide de la habitación; queda como TODO reducirlo.
8. **Bug de routing identificado pero no corregido:** la query “*the laptop in the bedroom*” se enruta a la sala de estar cuando los `room_scores` están casi empatados (`bedroom`=0.37 vs `living room`=0.35). El *room hint* explícito de la query no rompe el empate. Es un bug del selector de habitación, fuera del alcance del placer; se registra en `tools/HOWTO_validate_small_objects.md` como follow-up para `vlmaps/policy/strategic_policy.py`.

27 de abril de 2026 — Desbloqueo de laptop en el catálogo y criba del set de objetos

1. **Diagnóstico del bug de catálogo de laptop:** el placer registraba sistemáticamente `[skip]` no HSSD `template for object 'laptop'` en dos de las tres escenas. El CSV `object_categories_filtered.csv` contenía únicamente identificadores de la familia `Laptop_X`, que no llevan `.object_config.json` en HSSD. El filtro `_available_object_handles()` los descartaba a todos antes de llegar a la fase de elección, dejando la categoría vacía.
2. **Fix: *_EXTRA_CATALOG* aditivo:** se añade un diccionario `_EXTRA_CATALOG` en `tools/place_small_objects.py` con ocho hashes de portátiles reales (Toshiba Satellite, intel celeron 15.4, Computer Laptop, Sony Vaio, HP Pavilion, Toshiba L300D, MacBook Pro 17 y MacBook Pro 15.4) que sí traen el `.object_config.json` completo. `_load_object_catalog()` fusiona estos hashes a posteriori, respetando el filtro de disponibilidad en disco. El cambio es aditivo: el resto del catálogo CSV se carga igual.
3. **Reducción del *floor offset* a 30 cm:** el offset para *placements* sobre suelo (usado por `trash bin`) se baja de 80 cm a 30 cm. Ya no hay regresión teórica para el bin actual y se protege contra futuros *multi-floor placements*.
4. **Criba del catálogo de objetos pequeños:** tras revisar los resultados de `post_placement_full` y discutir las propiedades semánticas de cada *target*, se decide *quitar* `trash bin` y `kettle` y *añadir* `book`. El razonamiento:
 - `trash bin` aparece en cinco habitaciones (`kitchen`, `bathroom`, `bedroom`, `office`, `hallway`) sin prior dominante. Esto vuelve la búsqueda *room-aware* indistinguible de una búsqueda plana M2: el sistema acaba probando casi todas las habitaciones. Mide ruido del

detector, no calidad del razonamiento jerárquico.

- **kettle** comparte exactamente el mismo prior que **teapot** (`kitchen = 1.0`), morfología similar (asa + pico) y YOLOE los confunde sistemáticamente. Tener los dos en la batería no añade señal experimental e infla la matriz de confusión.
 - **book** aporta el caso multi-room interpretable (`office > bedroom > living`), forma distintiva, abundancia de *assets* con `.object_config.json` en HSSD y prior ya cargado en `room_priors.py`. Es justo el tipo de consulta donde el razonamiento jerárquico debe destacar frente al M2 plano.
5. **Specs actualizadas:** las tres `tools/eval_queries/specs/small_objects_*.json` pasan de 8 a 7 placements (50 → 45 queries efectivas). Distribución del nuevo **book**: en 102344193_0 y 102344280_0 sobre `table` en `bedroom`; en 108736884_177263634_4 sobre `table` en `living room` para diferenciarlo del `laptop` que ya está en `office`.
 6. **Documentación operativa:** se redacta `tools/HOWTO_validate_small_objects.md` con tres procedimientos progresivos (*quick check* de 1 min, *smoke test* de 5 min, *full eval* de ~30 min) y los criterios de fallo esperados, de modo que la validación post-placement se pueda repetir sin asistencia.
 7. **Aclaración metodológica para la fase posterior:** el régimen actual coloca cada objeto en su zona más probable (caso *happy path* room-aware). La métrica que cierra la hipótesis exigirá generar N variantes del mismo mapa con colocaciones plausibles distintas (botella en encimera vs. comedor vs. mesilla, libro en escritorio vs. mesilla vs. sofá) y comparar cómo degradan el sistema room-aware y el M2 plano cuando el objeto no está en su prior. Esa generación automática y la agregación cross-variant quedan como trabajo siguiente a `post_placement_v2`.

28 de abril de 2026 — Separación estable HSSD/ROS y arranque del Toyota HSR oficial en Gazebo

1. **Separación explícita de los dos frentes del proyecto:** se fija que `tfg-sim` conserva intacto el pipeline Habitat/HSSD (colección, evaluación, queries y métricas SR/CFR), mientras que `tfg-ros` concentra la migración ROS 1 Noetic + Gazebo. Esta separación evita que los experimentos HSSD se rompan al iterar sobre Gazebo, RViz, sensores o modelos del robot.
2. **Dos perfiles de robot en ROS:** se mantiene `hsr_proxy_gazebo_move_base.launch` como backend navegable ligero para validar `move_base`, `rosbridge` y `/vmap/navigation_result`. En paralelo se añade `hsr_gazebo_move_base.launch` para lanzar el Toyota HSR oficial público como `hsrb`, con mallas, árbol TF y sensores más fieles.
3. **Modelos oficiales públicos sin vendorear assets:** se incorporan localmente `hsr_meshes` y `hsr_description` bajo `ros_ws/src/external/`. La reproducción queda automatizada con `tools/fetch_hsr_official_models.py`, pero los repositorios externos se excluyen de Git para no subir mallas ni documentación de terceros al repositorio del TFG.
4. **Compatibilidad Noetic del Xacro oficial:** el modelo público requería adaptar sintaxis antigua de Xacro (`macro` → `xacro:macro`, `insert_block` → `xacro:insert_block`) y corregir nombres de transmisiones generados. El síntoma visible era que el HSR aparecía sin cabeza o con enlaces colapsados en el origen; tras los parches la cabeza queda situada correctamente y el modelo se puede inspeccionar en Gazebo.
5. **Topics HSR alineados con sensores reales:** la configuración ROS ya referencia `/hsrb/base_scan`, `/hsrb/odom_ground_truth`, `/hsrb/head_rgb_sensor/rgb/image_rect_color`, `/hsrb/head_rgb_sensor/hsrb/head_rgb_sensor/rgb/camera_info`, `/hsrb/head_rgb_sensor/depth_registered/camera_info`.

y `/hsrb/head_rgbd_sensor/depth_registered/rectified_points`. El siguiente checkpoint técnico no es mover todavía el robot físico, sino visualizar y validar estos topics en RViz.

6. **Próximos pasos:** primero se debe construir un panel RViz con `RobotModel`, `TF`, `laser`, `RGB`, `depth` y nube de puntos; después validar la cadena `map` → `odom` → `base_footprint` → `base_link` → `head_rgbd_sensor_rgb_frame`; después conectar `yoloe_verifier` a un topic `RGB` configurable; después resolver la movilidad del HSR oficial mediante controladores TMC o mantener el proxy como backend de navegación; y finalmente crear un *house world* Gazebo con habitaciones anotadas mediante centroides + Voronoi. Manipulación, MoveIt, brazo y grasping se posponen hasta que navegación y percepción sean estables.

4 de mayo de 2026 — Consolidación del backend ROS/HSR y aceleración gráfica

1. **HSR oficial como camino principal en Gazebo:** el frente ROS deja de apoyarse en el robot *proxy* como referencia de uso diario. Se estabiliza el arranque del Toyota HSR oficial en mundos simples (`hsr_gazebo_walls.launch`, `hsr_gazebo_simple.launch` y variantes `indoor`), manteniendo `tfg-sim` intacto como backend Habitat/HSSD del TFG.
2. **Sensores visibles en RViz:** queda operativo un panel RViz capaz de mostrar `RobotModel`, `TF`, `LaserScan`, `RGB-D`, `depth`, nube de puntos y marcadores auxiliares. Esto convierte la validación del HSR en una inspección visual reproducible, sin depender sólo de logs o topics aislados.
3. **Teleoperación estándar ROS:** se sustituye el teleop casero por `teleop_twist_keyboard`, remapeado a `/hsrb/command_velocity`. Con ello la teleoperación pasa a usar una herramienta estándar del ecosistema ROS 1 y deja de depender de un script ad hoc del proyecto.
4. **Diagnóstico y fix de rendimiento gráfico en tfg-ros:** el contenedor ROS estaba renderizando Gazebo con `Mesa llvmpipe` (software rendering por CPU). Se habilita acceso NVIDIA en `docker-compose.yml` para `tfg-ros`, con capacidades gráficas adecuadas, y se verifica que tanto Gazebo como OpenGL pasan a usar la GPU NVIDIA real. El impacto práctico es que Gazebo GUI y RViz dejan de estar penalizados por rasterización software.
5. **Escenarios mínimos para validación:** además de los mundos internos de Gazebo, se añade un mundo mínimo con paredes (`minimal_walls.world`) para depurar navegación, sensores y render en el caso más simple posible. Esto da un banco estable para comprobar cámara, lidar, odometría y teleop antes de saltar a mundos más complejos.
6. **Estado al cierre del día:** la arquitectura queda ya dividida en dos caminos claros. `tfg-sim` conserva la lógica semántica, HSSD, métricas y evaluación; `tfg-ros` dispone de Gazebo, RViz, HSR oficial, teleop estándar y aceleración GPU funcional. El siguiente paso ya no es infraestructura básica, sino seguir refinando la conexión entre ambos contenedores y el uso del backend ROS como ejecutor de goals semánticos.

6 de mayo de 2026 — Regiones LabelMe y exploración activa de habitaciones en Habitat/HSSD

1. **Regiones manuales como fuente prioritaria:** el flujo HSSD acepta ahora mapas de habitación creados con LabelMe. Si existe `room_map/` para una escena, `LabelMeRoomProvider` se usa por delante de las regiones nativas de HSSD. Esto permite definir habitaciones como zonas navegables reales, no sólo como centroides o polígonos semánticos imperfectos.

2. **Escenas preparadas:** las escenas 102344193_0 y 102344280_0 ya tienen regiones LabelMe convertidas y cargables por `interactive_object_nav.py`. La escena 108736884_177263634_4 se retira del menú HSSD para evitar confusión con la variante 108736884_177263634_0.
3. **Menos escaneos redundantes:** se añade memoria de zonas de escaneo por episodio. Si el robot vuelve a una posición situada a menos de `VLMAPS_SCAN_DEDUP_RADIUS_M` metros de una zona ya escaneada, se evita repetir el barrido local. El valor por defecto es 1,5 m.
4. **Exploración de habitación con YOLOE activo:** al activar `VLMAPS_ROOM_EXPLORATION=1`, si falla el candidato principal y fallan los muebles alternativos de la habitación, el sistema genera puntos de paso dentro de la región LabelMe, los ordena por proximidad y navega por ellos con YOLOE ejecutándose durante el movimiento. Si YOLOE dispara con umbral bajo, el robot pausa la ruta, centra la vista, se acerca y confirma con umbral alto. Si la confirmación falla, registra la zona como falsa alarma y continúa explorando.
5. **Parámetros de control:** la exploración queda acotada por `VLMAPS_EXPLORE_MAX_POINTS` (8 por defecto), `VLMAPS_EXPLORE_TIMEOUT_S` (60 s), `VLMAPS_YOLOE_ROOM_LOW_THRESH` (0,40), `VLMAPS_YOLOE_CONFIRM_THRESH` (0,65) y `VLMAPS_MAX_APPROACH_ATTEMPTS` (3). Por defecto la exploración está desactivada para no alterar experimentos previos.
6. **Telemetría nueva:** `SearchState` y `[eval-summary]` incluyen `entered_zone_exploration`, `n_exploration_points_planned`, `n_exploration_points_visited`, `n_unique_scan_zones`, `n_low_conf_detections`, `n_approach_attempts`, `n_false_positive_zones`, `continuous_yoloe_triggered` y `final_success_source`. Estas métricas permiten separar búsquedas resueltas por candidato directo, por mueble alternativo o por exploración de la habitación.
7. **Prueba manual desde el menú:** abrir menu → VLMAPS pipeline → HSSD → x) LLM navigation + room exploration. Esa opción activa `VLMAPS_ROOM_EXPLORATION=1`, pregunta los límites de exploración y lanza `interactive_object_nav.py` sin tener que preparar variables manualmente. La opción 1) Interactive LLM navigation mantiene el comportamiento clásico sin exploración activa.
8. **Doble capa LabelMe/Voronoi:** se separan dos usos que antes estaban mezclados. `room_map.npy` conserva solo el suelo navegable marcado manualmente y se usa para movimiento, puntos de paso y límites duros de exploración. La nueva capa `room_voronoi.npy` expande cada región navegable hacia las celdas no etiquetadas cercanas y se usa solo para asignar muebles y candidatos VLMa a una habitación. Así una mesa situada sobre una celda ocupada puede pertenecer a `living_room`, mientras el robot sigue caminando únicamente por suelo navegable.
9. **Búsqueda confinada por habitación:** en órdenes como `go to the living room and find the drill`, la habitación explícita pasa a ser una restricción dura. El robot debe entrar en la zona LabelMe de `living_room`, filtrar muebles mediante Voronoi y explorar solo puntos de paso dentro de la máscara navegable de esa habitación.

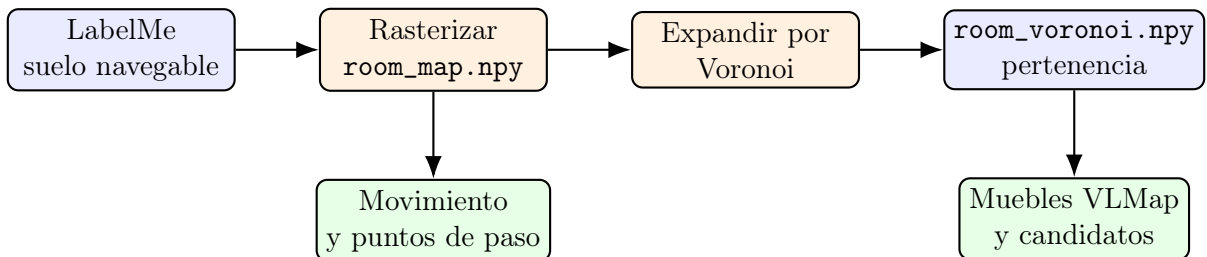


Figura 8: Arquitectura de doble capa: LabelMe define dónde puede pisar el robot; Voronoi define a qué habitación pertenecen los muebles y candidatos.

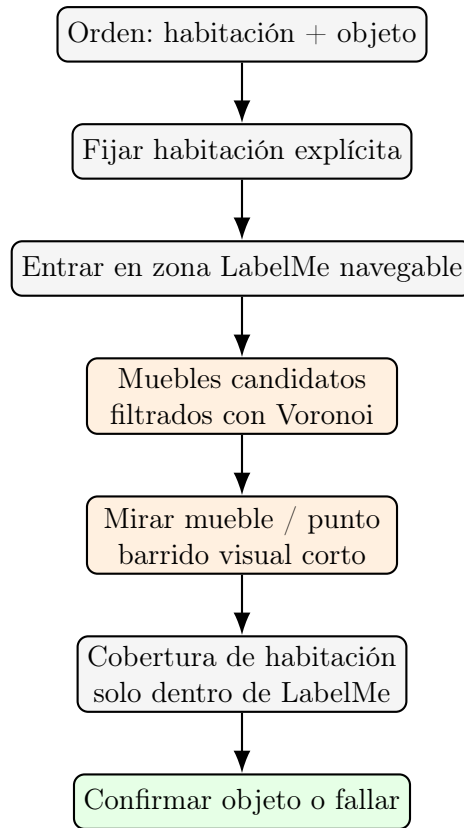


Figura 9: Flujo previsto para búsquedas confinadas: la habitación explícita limita tanto la exploración como los muebles inspeccionables.

7 de mayo de 2026 — Corrección de LabelMe/Voronoi y validación de regiones

1. **Sincronización automática antes de navegar:** las opciones 1) `Interactive LLM navigation` y x) `LLM navigation + room exploration` reconvierten automáticamente el JSON de LabelMe si existe en `/workspace/annotations/room_labels`. Esto evita que el sistema cargue un `room_map` antiguo cuando se han añadido o renombrado regiones desde LabelMe.
2. **Validación de etiquetas:** al arrancar con `VLMAPS_ROOM_EXPLORATION=1`, el sistema compara las etiquetas del JSON de LabelMe con las habitaciones cargadas por `LabelMeRoomProvider`. Si falta alguna región, aborta con un mensaje explícito en vez de explorar con un mapa incompleto.
3. **Voronoi limitado por paredes:** se sustituye la expansión euclídea global por una expansión conectada desde las regiones LabelMe dentro de un dominio interior visible. El dominio se obtiene de `topdown_labeled.png`, `topdown_rgb.png` y `obstacle_map.png`, conserva solo componentes conectados que tocan alguna región LabelMe y no permite asignar celdas fuera de ese dominio. La capa resultante sigue siendo solo de pertenencia semántica; el robot continúa caminando únicamente por `room_map.npy`.
4. **Artefactos de depuración:** cada conversión guarda `room_voronoi_domain.png` y `room_voronoi_viz.png`, además, el arranque de x guarda `debug/room_debug_overlay.png`. Estas imágenes permiten comprobar visualmente si la expansión se queda dentro de la vivienda y si los muebles caen bajo la habitación correcta.
5. **Validación realizada:** se regeneraron las tres escenas HSSD etiquetadas. Resultado: 102344193_0 carga 7/7 etiquetas, 102344280_0 carga 12/12 y 108736884_177263634_0 carga 15/15. En las tres escenas, `outside_domain=0`: no hay celdas Voronoi asignadas

fuera del dominio interior generado.

6. **Acercamiento antes de confirmar:** una detección visual ya no se acepta como éxito final por confianza alta desde lejos. Cualquier disparo de YOLOE pasa primero por una fase común de acercamiento: orientar la cámara, intentar un acercamiento planificado al mueble soporte cuando existe, avanzar sólo si la siguiente celda es navegable y permanece dentro de la habitación fijada, y repetir la confirmación con umbral alto desde cerca.
7. **Circuito fijo por habitación con disparo en ruta:** la opción `x` mantiene un número fijo de puntos por habitación; por defecto, 8. La generación de puntos ya no usa una rejilla rectangular sobre la caja envolvente de la habitación: ahora agrupa directamente las celdas navegables LabelMe reales con una selección tipo *k-medoids*, prueba tres niveles de candidatos (margen preferido, margen relajado y cualquier celda navegable), aplica una separación mínima configurable (`VLMAPS_EXPLORE_POINT_MIN_SEP_M`, 0,90 m por defecto) y ordena el resultado por vecino más cercano desde la pose actual. Esto evita que una región no convexa concentre todos los puntos en el centro o deje sin cubrir esquinas útiles. La opción `x` también activa `VLMAPS_EXPLORE_DEBUG=1`, guardando una imagen por exploración en `debug/zone_explore_<room>_<target>.png` con máscara, candidatos y circuito elegido. Mientras el robot recorre ese circuito, YOLOE sigue vigilando con umbral bajo 0.40: si aparece una detección, la ruta se pausa, se abandona temporalmente el punto del circuito y el robot proyecta una meta navegable en la dirección de la cámara que acaba de ver el objeto. La confirmación se hace con una sesión YOLOE separada a 0.80, de modo que el umbral bajo de aviso y el umbral alto de confirmación no se pisan. Si no hay disparo en ruta, al llegar a cada punto hace un barrido lateral $\pm 30^\circ$ y pasa al siguiente.

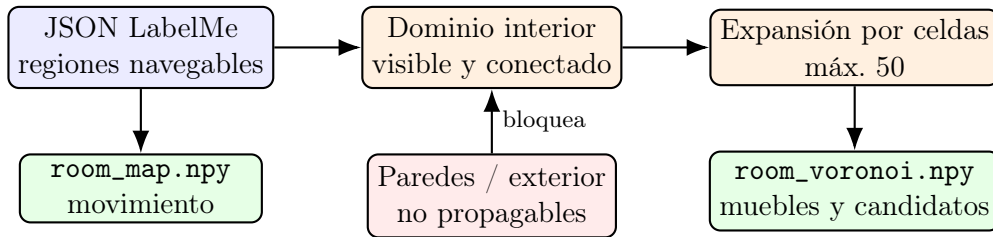


Figura 10: Voronoi corregido: las regiones LabelMe se expanden solo por el dominio interior conectado, de modo que la pertenencia de muebles no atraviesa paredes ni exterior.

11 de mayo de 2026 — Ajuste de navegación estrecha y acercamiento visual

1. **Inflación de planificación reducida:** el mapa usado para el grafo de planificación pasa de 3 a 2 iteraciones de dilatación por defecto (~ 10 cm por lado con celdas de 5 cm). Esto permite entrar en pasos más estrechos sin desactivar el escudo de ejecución, que sigue usando el mapa bruto de navegación. El valor queda parametrizado con `VLMAPS_PLAN_DILATION_ITERS`; para volver al comportamiento anterior basta lanzar con valor 3.
2. **Corrección del giro hacia objetivo:** la rutina `face_toward_pos()` se alinea con la convención angular real observada en Habitat: 0° mira hacia el norte del mapa y, en este wrapper, `turn_left` incrementa el ángulo almacenado. Antes mezclaba otra convención y podía orientar la cámara hacia el lado opuesto del objeto detectado.
3. **Acercamiento más permisivo y trazable:** los puntos de acercamiento a una detección visual usan ahora una holgura mínima de 2 celdas por defecto, configurable con `VLMAPS_APPROACH_MIN_CLEARANCE_CELLS`. Si el mapa inflado no ofrece ningún punto viable, se intenta una segunda selección sobre el mapa bruto navegable y se conserva el filtro de no salir de la región LabelMe fijada.

4. **Proyección visual menos conservadora:** cuando YOLOE dispara con umbral bajo durante la exploración, la celda objetivo proyectada en dirección de la cámara se calcula sobre el mapa bruto navegable. Esto evita cortar la proyección demasiado pronto cerca de muebles, que era una de las causas de acercamientos que apenas se movían.
5. **Recuperación ante fallo de sesión YOLOE:** si durante el centrado, el acercamiento o la confirmación final aparece un **Broken pipe**, se reabre la sesión de YOLOE y se reintenta una vez antes de descartar el acercamiento. El objetivo sigue siendo el mismo: detección lejana a 0,40 sólo dispara acercamiento; éxito final únicamente con confirmación cercana a 0,80.

12 de mayo de 2026 — Servo visual antes de planificación en detecciones YOLOE

1. **Acercamiento guiado por la detección:** cuando YOLOE detecta un objeto con umbral bajo durante la exploración, el robot ya no gira primero hacia una celda proyectada ni navega directamente a un *standoff* lateral. Primero centra la caja de YOLOE y avanza por pasos seguros mientras respeta obstáculos y la región LabelMe activa. La planificación A* queda como respaldo si el acercamiento frontal no logra avanzar.
2. **Menos cambios de objetivo durante el acercamiento:** el flujo anterior podía perder la botella detectada, girar hacia otra zona y enganchar otra botella parecida. El nuevo orden mantiene la referencia visual original como prioridad y evita preplanificar hacia muebles o celdas estimadas antes de intentar el acercamiento visual.
3. **Confirmación estricta:** la detección lejana a 0.40 sigue siendo sólo un disparador. La búsqueda únicamente se marca como exitosa tras confirmación cercana con el umbral alto configurado, por defecto 0.80.
4. **Resumen de consola legible:** el bloque JSON completo [eval-summary] deja de imprimirse en modo interactivo por defecto. En su lugar aparece un resumen corto con objetivo, habitación, puntos explorados, detecciones de bajo umbral, acercamientos y falsas alarmas. Para evaluaciones automáticas se conserva el JSON activando `VLMAPS_EVAL_SUMMARY_JSON=1`; en modo headless sigue activo por defecto.

12 de mayo de 2026 — Puntos de exploración adaptativos por región

1. **Modo automático por defecto:** el flujo x ya no fuerza siempre 8 puntos por habitación. Si se deja el valor por defecto `auto`, el sistema calcula el número de puntos según el área navegable LabelMe de la región: una habitación de tamaño de referencia recibe 8 puntos, una de doble tamaño tiende a 16 y una de tres cuartos tiende a 6.
2. **Límites prácticos:** para evitar rutas demasiado largas o zonas pequeñas mal cubiertas, el modo automático queda acotado por defecto entre 4 y 20 puntos, configurable con `VLMAPS_EXPLORE_MIN_POINTS` y `VLMAPS_EXPLORE_MAX_POINTS_CAP`. El número base por región de referencia se controla con `VLMAPS_EXPLORE_POINTS_PER_REFERENCE`.
3. **Corrección por forma:** además del área, se aplica un pequeño factor de forma. Las regiones alargadas o con baja ocupación dentro de su caja envolvente reciben algún punto adicional porque suelen tener brazos, esquinas o pasillos que un muestreo puramente proporcional puede ignorar.
4. **Reproducibilidad:** si se escribe un número en el menú x, se desactiva el modo automático y se mantiene exactamente ese número de puntos, como antes. Esto permite repetir experimentos con una configuración fija cuando interese comparar resultados.

12 de mayo de 2026 — Ajuste de puntos y acercamiento a detecciones

1. **Menos puntos por defecto:** el modo automático deja de usar la mediana de habitaciones como unidad principal, porque en escenas con habitaciones pequeñas hacía que una cocina de tamaño medio escalase a 12 puntos. La referencia pasa a ser física, 8 m^2 , con 8 puntos base, mínimo 4 y máximo 16. En la práctica, una cocina de unos $7\text{--}8\text{ m}^2$ vuelve a generar alrededor de 8 puntos.
2. **Centrado visual con memoria:** durante el centrado YOLOE se guarda la última caja detectada. Si el último giro hace perder el objeto, se deshace ese giro una vez y el acercamiento conserva la caja anterior como pista visual. Esto evita que el robot corte el acercamiento tras un único giro demasiado agresivo.
3. **Respaldo planificado más cercano:** si el acercamiento visual no alcanza la confirmación de 0,80, el respaldo A^* ya no prioriza el punto de mayor holgura alrededor de la detección. Ahora prioriza el punto navegable viable más cercano a la celda proyectada del objeto, y al llegar se orienta explícitamente hacia esa celda antes de volver a confirmar.
4. **Persistencia tras avance parcial:** aunque el servo visual haya avanzado algo, si no confirma desde cerca se intenta igualmente el respaldo al punto cercano al objeto. La detección de 0,40–0,79 sigue siendo sólo un disparador; el éxito continúa requiriendo confirmación cercana con el umbral alto.

27 de mayo de 2026 — Robustez de exploración, detecciones persistentes y filtrado navegable

Tras las pruebas con `mug`, `basketball`, `teddy bear`, `bottle` y `laptop` en las escenas HSSD, se detectaron tres fallos prácticos: puntos de exploración que visualmente caían en la habitación pero no eran alcanzables por el robot, detecciones de baja probabilidad que interrumpían demasiado pronto el circuito de cobertura, y una persistencia visual que no debía dibujar cajas propias sino conservar exactamente la anotación producida por YOLOE.

1. **Persistencia visual basada exclusivamente en YOLOE:** cuando YOLOE devuelve una detección positiva se guarda el frame anotado por el propio detector (`ann_rgb`). Si en los siguientes frames la detección parpadea brevemente, la interfaz reutiliza ese frame anotado. No se dibujan cajas adicionales desde la capa de navegación. La marca persistente, por tanto, es la misma caja azul/verde generada por YOLOE, con su etiqueta y confianza originales.
2. **Caducidad espacial de la marca:** la persistencia anterior no equivale a mostrar evidencia obsoleta. Si el robot gira demasiado, se desplaza a otra pose o la detección envejece, la vista vuelve a la cámara RGB en vivo. Esto evita el caso observado en el que una caja permanecía en pantalla aunque el objeto ya no estuviese visible.
3. **Filtrado de puntos realmente navegables:** antes de aceptar un punto de exploración se valida la ruta completa con el mismo mapa usado para planificar. El punto se descarta si cae en obstáculo, si su holgura es insuficiente, si A^* no llega al objetivo, si la ruta cruza obstáculos o si sale indebidamente de la máscara de habitación. En los logs aparece como `Puntos descartados por no alcanzables`.
4. **Sobremuestreo y selección posterior:** la generación ya no pide exactamente N puntos y los acepta sin más. Primero genera más candidatos de los necesarios, después filtra por alcanzabilidad real y finalmente conserva los mejores N . Esto permite mantener cobertura aunque varios puntos aparentemente buenos resulten imposibles para la huella del robot.
5. **Rutas que empiezan fuera y reentran:** si el robot viene de otra habitación, una ruta válida puede empezar fuera de la máscara objetivo y entrar inmediatamente en ella. El vali-

dador permite ese caso, pero rechaza trayectorias que atraviesan o abandonan la habitación una vez dentro.

6. **Continuación del circuito tras detección no confirmada:** una detección baja o incluso una alta probabilidad no confirmada desde cerca ya no aborta la exploración de la habitación. El sistema intenta confirmar desde el punto actual; si no lo consigue, continúa hacia el resto de puntos para buscar otro ángulo. Esto corrige casos como *mug* y *basketball*, donde una vista lateral débil no bastaba para confirmar.
7. **Mejora de soportes semánticos VLMaps:** se amplían los muebles usados para focalizar la exploración. Además de *table*, *counter*, *shelf* y *cabinet*, se incluyen *bed*, *sofa* y *chair*. Esto mejora objetos como *basketball* y *teddy bear*, que no siempre aparecen cerca de superficies de cocina.
8. **Alias y priors nuevos:** se añaden alias en inglés y español para *teddy bear* (peluche, oso de peluche, *stuffed bear*) y se ajustan los priors de habitación para *ball/basketball* y *teddy bear*. La intención es que el room-first no busque únicamente donde el objeto es visible para YOLOE, sino donde domésticamente tiene sentido.

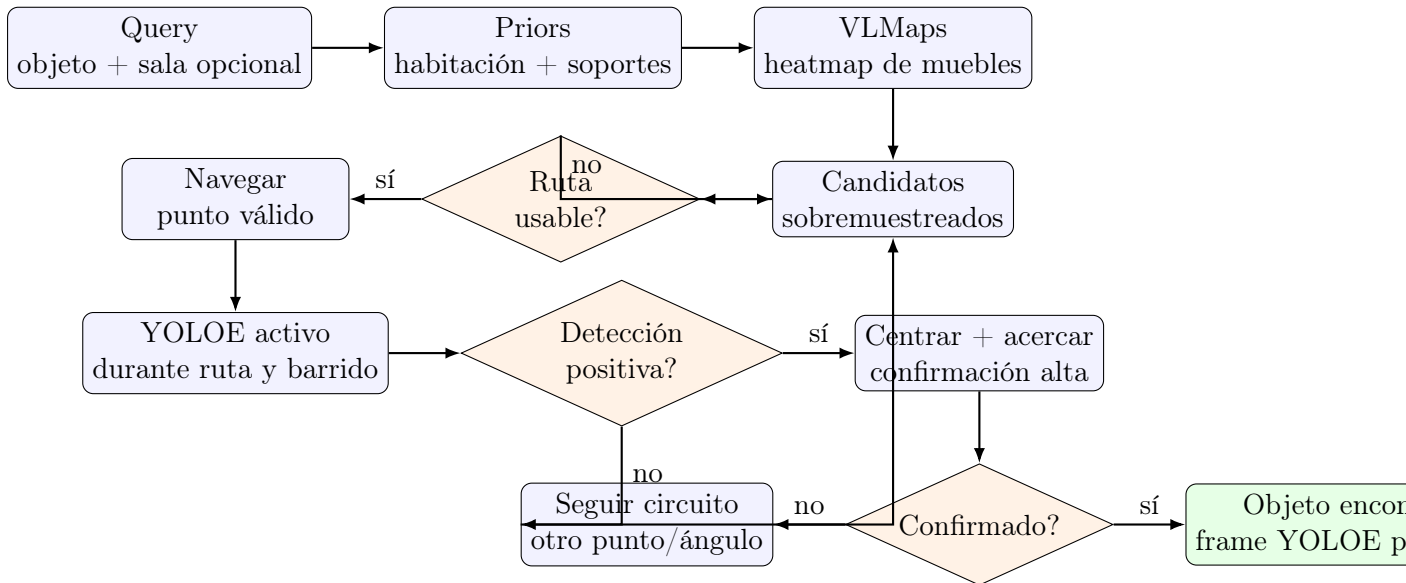


Figura 11: Flujo robusto tras la corrección del 27 de mayo: VLMaps propone zonas semánticas, el planificador descarta puntos no alcanzables, YOLOE verifica durante navegación y sólo la anotación original del detector se mantiene de forma temporal en pantalla.

28 de mayo de 2026 — Smoke real de doble contenedor (tfg-sim → tfg-ros)

Cerrada la navegación base del Toyota HSR en Gazebo (Fase 1), se valida por primera vez el *contrato* completo entre los dos contenedores (Fase 2): desde **tfg-sim** se pide una navegación a **tfg-ros** sin conocer nada del robot salvo el contrato ROS, y vuelve un resultado con token, éxito/fracaso y habitación. La regla arquitectónica se mantiene: la lógica de decisión sigue en **tfg-sim** y ROS actúa sólo como *backend* de navegación y sensores.

Comandos de la prueba. La prueba usa tres terminales. La primera arranca el robot, su navegación y el puente *rosbridge* dentro de **tfg-ros**:

```
# Terminal 1 (tfg-ros): Gazebo + move_base + RViz + rosbridge
roslaunch vlmap_bringup hsr_gazebo_move_base.launch \
  gui:=true use_rviz:=true use_rosbridge:=true
```

La segunda observa el topic neutral de resultado dentro de `tfg-ros`:

```
# Terminal 2 (tfg-ros): observador del resultado
rostopic echo /vlmap/navigation_result
```

La tercera envía el `SemanticGoal` desde `tfg-sim` a través de `rosbridge`, usando la herramienta `send_ros_semantic_goal.py` y el `RosNavigationBackend`:

```
# Terminal 3 (tfg-sim): enviar el goal por rosbridge
python tools/send_ros_semantic_goal.py \
    --host tfg-ros --x 0.50 --y 0.50 --yaw 0.50 \
    --room office --object laptop --timeout 30
```

Resultado observado. El robot gira y avanza hasta la pose pedida sin movimientos erráticos (gracias al `cmd_vel smoother` y al giro limitado a `max_vel_theta = 0,22`). El recorrido del mensaje fue el esperado:

1. `tfg-sim` publica el goal y `tfg-ros` recibe el `SemanticGoal` con su `navigation_token`.
2. se reexpide a `/move_base_simple/goal` como `PoseStamped` con `yaw` real.
3. `move_base` planifica, navega y emite `/move_base/result` con `status = 3 (SUCCEEDED)`.
4. el puente devuelve `/vlmap/navigation_result` con el mismo token, `success = true` y `actual_room`.

El JSON de resultado confirma el cierre del contrato (token idéntico de ida y vuelta, `result_status_code = 3`). En esta corrida `room_source = requested_echo`: la habitación devuelta es todavía un eco de la pedida, porque el contexto semántico vivo aún no se publicaba. Esa pieza es justo la siguiente (Fase 3): `tfg-sim` publica `/vlmap/room_context` (latched, JSON en `std_msgs/String`) con habitaciones y centroides, y `tfg-ros` resuelve `actual_room` desde la pose final real (`room_source = pose_resolved`).

28 de mayo de 2026 (segunda iteración) — Contexto semántico vivo (Fase 3) y giro más lento

Cerrado el contrato de navegación (Fase 2), se implementa la **Fase 3**: `tfg-sim` pasa a publicar el contexto semántico de habitaciones en vivo, de modo que `tfg-ros` resuelve la habitación real desde la pose final del robot en lugar de hacer eco de la pedida. Se respeta la regla arquitectónica: el contexto viaja como *datos* (no como geometría de Gazebo), así que el mundo de simulación robótica no necesita cargar las habitaciones.

1. **Transporte elegido**: topic *latched* `/vlmap/room_context` con JSON en `std_msgs/String` (mismo patrón que `SemanticGoal` y `navigation_result`). Los mapas/heatmaps, si se envían, viajan como *referencias* a `/shared/` (bind mount), nunca embebidos, para no saturar `rosbridge`.
2. **Publicador (lado tfg-sim)**: nueva clase `RoomContextPublisher` en `tfg_nav_contracts`, simétrica a `RosNavigationBackend` (roslibpy sobre `rosbridge`, sin dependencia de `rospy`). `build_payload()` arma `{rooms, priors, maps}` y `publish()` lo emite latched.
3. **Resolución por pose (lado tfg-ros)**: se añade `DynamicRoomContext.room_at_pose(x, y, max_distance)`, que devuelve la habitación cuyo centroide está más cerca de la pose final. El nodo `vlmap_navigation_result_node` se suscribe a `/vlmap/room_context` (latched) y a la pose del robot (`/amcl_pose`); al cerrar un goal resuelve `actual_room` desde la pose real.
4. **Trazabilidad**: `build_navigation_result_payload` acepta `resolved_room`; cuando hay contexto vivo el resultado lleva `room_source=pose_resolved` y guarda `requested_room`; sin contexto mantiene el fallback `room_source=requested_echo`.

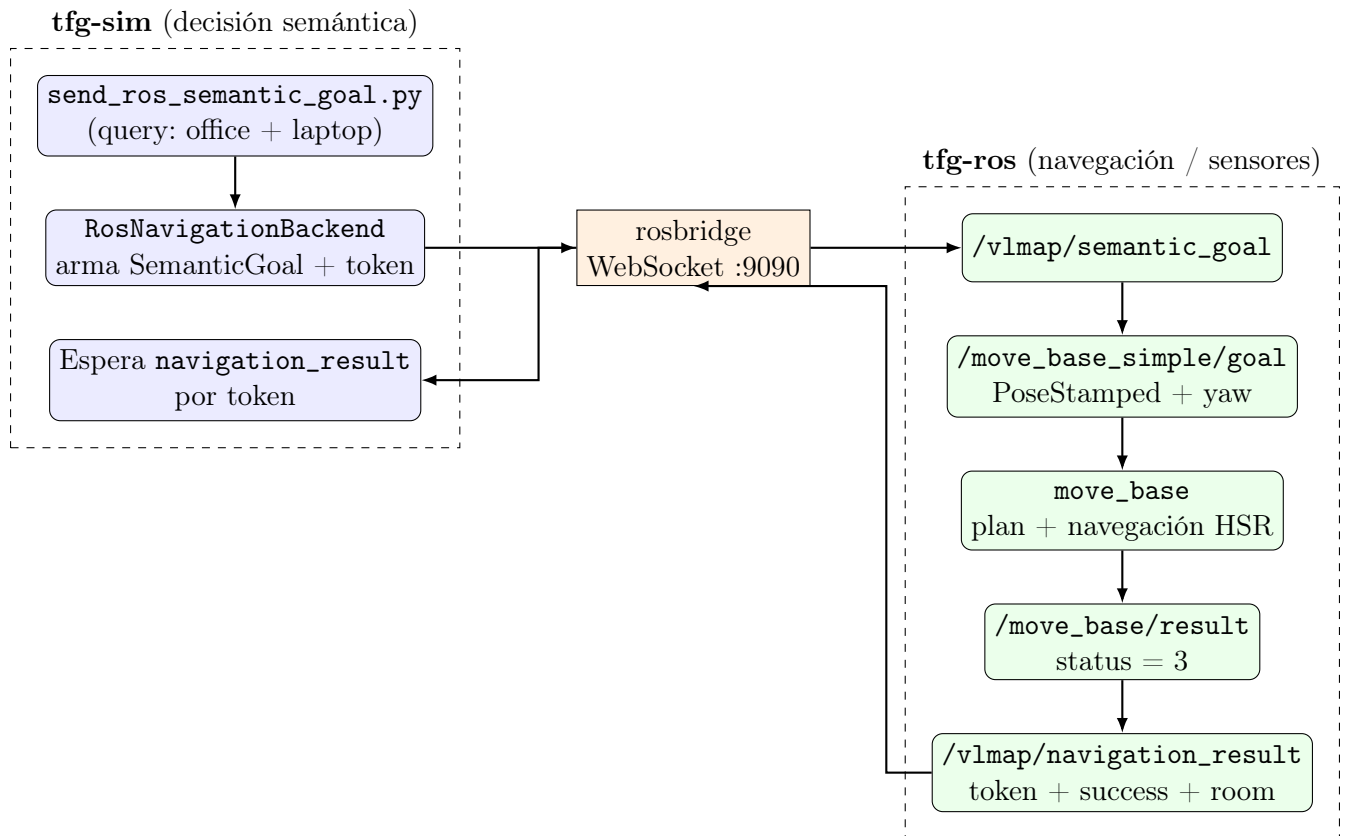


Figura 12: Flujo del smoke de doble contenedor (28 de mayo). `tfg-sim` decide y publica un `SemanticGoal` por `rosbridge`; `tfg-ros` lo reexpide a `move_base`, navega el HSR y devuelve `navigation_result` con el mismo token. La inteligencia no se porta a ROS: éste actúa sólo como backend de navegación.

5. **Herramienta de disparo:** `tools/publish_room_context_live.py`, simétrica a `send_ros_semantic_goal.py`.
Modos: `-demo` (contexto de 3–4 salas con coordenadas en el frame `/map`), `-context-file` (JSON listo) y `-scene` (centroides de rejilla de `room_map/regions.json`). Por defecto mantiene la conexión abierta para que el topic latched siga disponible a suscriptores tardíos; `-once` publica y sale.
6. **Tests:** nuevo `tests/test_room_context_live.py` (nearest-centroid, round-trip publicador↔parser, override de `actual_room`). Total **27 pruebas en verde**.

Smoke de Fase 3. Con `tfg-ros` arrancado, desde `tfg-sim`:

```
# 1) publicar contexto vivo (se queda latched)
python tools/publish_room_context_live.py --host tfg-ros --demo
# 2) mandar el goal a una sala conocida del contexto
python tools/send_ros_semantic_goal.py \
    --host tfg-ros --x 2.5 --y 0.0 --yaw 0.0 \
    --room office --object laptop --timeout 30
```

Criterio de cierre: el resultado pasa de `room_source="requested_echo"` a `"pose_resolved"` y `actual_room` se deriva de la pose final real, eliminando la dependencia de centroides demo-estáticos.

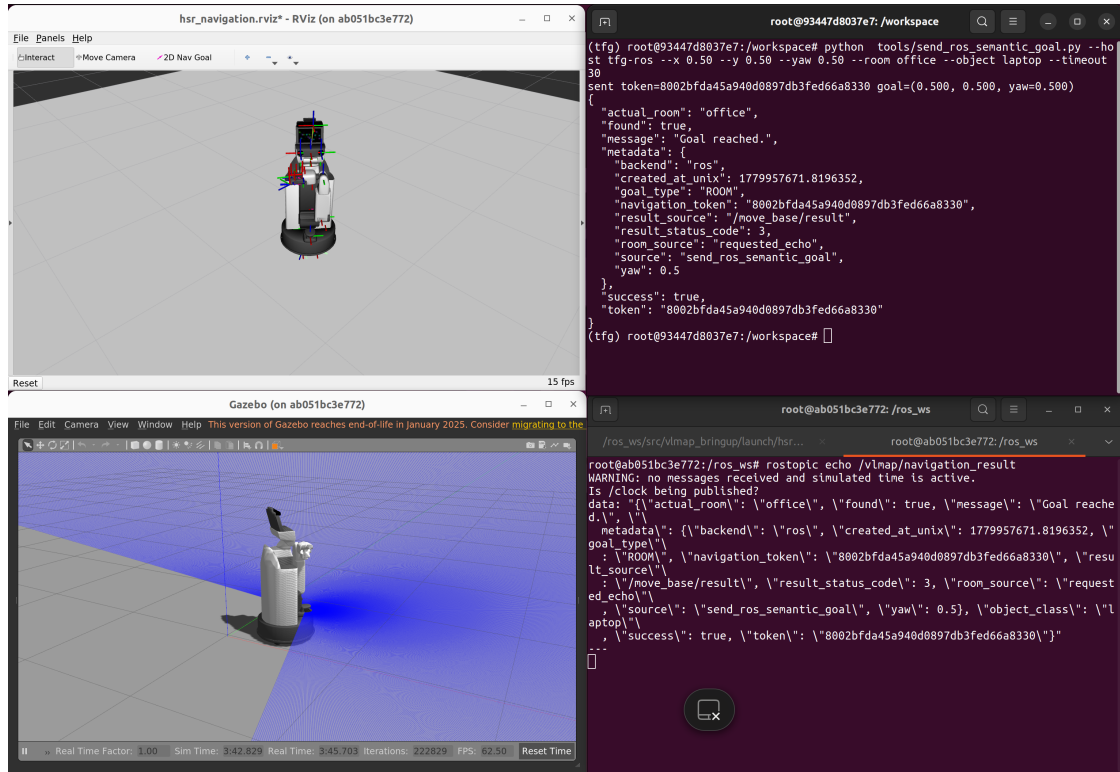


Figura 13: Evidencia de la prueba: RViz (arriba izq.) y Gazebo (abajo izq.) con el Toyota HSR; a la derecha, tfg-sim enviando el goal (token=8002bfda..., goal=(0,5,0,5,yaw=0,5)) y tfg-ros mostrando /vmap/navigation_result con success=true, result_status_code=3 y actual_room=office.

Cierre verificado y ajuste de la fuente de pose. En la primera ejecución el resultado seguía marcando `requested_echo` pese a publicar el contexto. La causa: el `launch hsr_gazebo_move_base` no incluye `AMCL` ni `map_server` (sólo un TF estático `map→odom` identidad), de modo que `/amcl_pose` nunca se publica y la pose del robot quedaba a `None`. Se corrige apuntando la fuente de pose por defecto a `/odom` (`nav_msgs/Odometry`): como `map→odom` es identidad, las poses de `/odom` ya están en el frame `map`. Un parámetro `~robot_pose_type` (`odom|amcl|pose_stamped`) mantiene el soporte para configuraciones con `AMCL`. Tras el ajuste, el `smoke` de doble contenedor devuelve `room_source=pose_resolved` y `actual_room` resuelto desde la pose final. **Fase 3 cerrada.**

La figura 14 resume el mecanismo: el contexto vivo (`latched`) y la pose del robot se combinan en el nodo de resultado para producir la habitación real, con *fallback* explícito y trazable al eco cuando no hay contexto.

Giro más lento (anti-convulsión). En las pruebas el HSR aún “convulsionaba” levemente al girar con `max_vel_theta = 0,22`. Se reduce de nuevo, sin volver al 0,12 que resultó demasiado lento: `max_vel_theta = 0,16`, `acc_lim_theta = 0,20`, `min_in_place_vel_theta = 0,06`, y en el suavizador `cmd_vel_max_angular = 0,16` con `cmd_vel_max_angular_accel = 0,10` (la rampa angular más baja es la que de verdad quita el tirón).

28 de mayo de 2026 (tercera iteración) — Fase 4: percepción en Habitat (Opción A)

Decidida la arquitectura de percepción: **Opción A**. ROS navega y los sensores físicos del HSR no se usan para verificar; la verificación visual la sigue haciendo Habitat, renderizando la pose

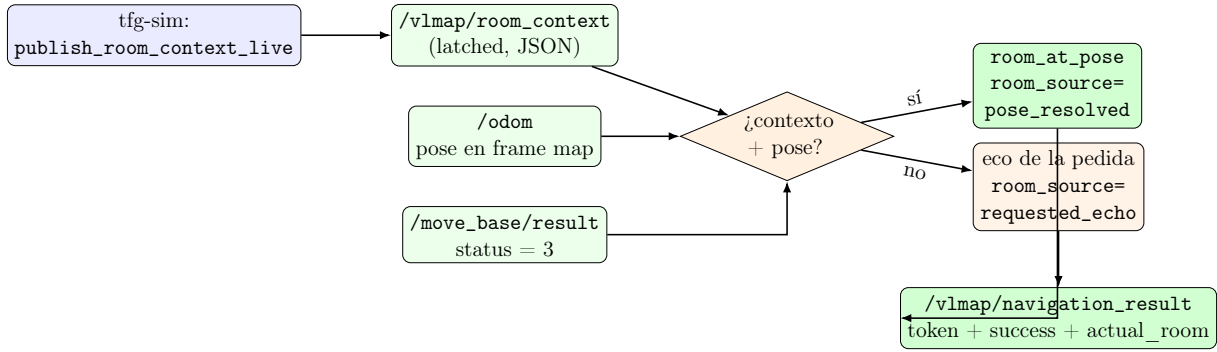


Figura 14: Resolución de `actual_room` en Fase 3. El contexto semántico vivo (latched) y la pose real del robot (`/odom`, `frame map`) se combinan en el nodo de resultado: si ambos están presentes, `room_at_pose` devuelve la habitación por centroide más cercano (`pose_resolved`); si no, se mantiene el eco trazable de la habitación pedida (`requested_echo`).

final donde sí están los objetos ya colocados (botella, taza, baloncesto, peluche, portátil). Así no se porta percepción a ROS y se reutiliza todo el trabajo de colocación. La alternativa (mundo espejo en Gazebo con los objetos reconstruidos en SDF/URDF) se descarta por coste y por romper la regla arquitectónica.

1. **La pose final viaja en el resultado:** el nodo de ROS guarda la pose del robot como (`x`, `y`, `yaw`) (`yaw` extraído del cuaternión) y `build_navigation_result_payload` la registra en `metadata.final_pose`. Así `tfg-sim` recibe la pose sin una segunda suscripción.
2. **Transformación de frames:** nueva clase `Rigid2D` (`tx`, `ty`, `theta`, `scale`) que expresa `T_habitat_from_map`. Para el Gazebo vacío actual ambos frames coinciden, así que la identidad es el valor por defecto; sólo hará falta una transformación no trivial cuando Gazebo cargue un mapa con origen/orientación distintos.
3. **PoseSyncBridge:** dado un `navigation_result`, extrae `final_pose`, la transforma al frame de Habitat y llama a un `verifier` inyectado (`verifier(pose, result)`). En producción ese callback teletransporta el agente de Habitat a esa pose y ejecuta YOLOE; en los tests es un *fake*. La lógica de transporte y transformación es pura y sin dependencia de ROS ni Habitat.
4. **Tests:** nuevo `tests/test_pose_sync.py` (identidad, traslación, rotación, inverso *round-trip*, extracción de pose y *hook* del verifier). Total **38 pruebas en verde**.

La figura 15 muestra el bucle completo previsto para la demo (Fase 5): la decisión semántica y la verificación viven en `tfg-sim`; ROS sólo navega y devuelve la pose real, que el `PoseSyncBridge` re proyecta a Habitat para que YOLOE confirme.

7 de junio de 2026: validación semántica multiclase del VLMa ROS/Gazebo

Se ha cerrado la validación del mapa construido desde una captura RGB-D del HSR en Gazebo. El resultado principal es que el fichero `vmaps.h5df` generado por el flujo ROS/Gazebo tiene el mismo formato operativo que los mapas HSSD que ya funcionaban en `tfg-sim`; el problema observado no estaba en la construcción del mapa, sino en la forma de medirlo.

El diagnóstico inicial usaba el score crudo de una sola categoría. Ese criterio no es válido para evaluar VLMa porque las puntuaciones CLIP de categorías aisladas son muy colineales: dos mapas de una única clase pueden correlacionar casi igual aunque describan zonas distintas. La validación nueva replica el uso nativo de VLMa: se inicializa el vocabulario completo con `init_categories(mp3dcat)`, se añaden las categorías solicitadas que no están en MP3D y cada

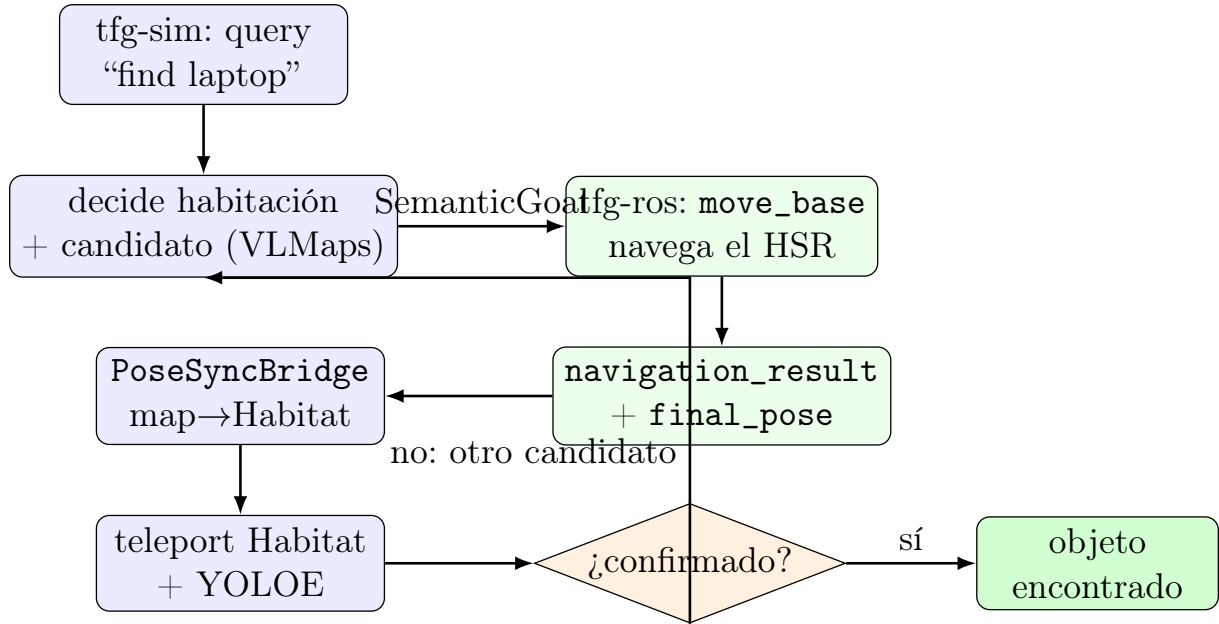


Figura 15: Bucle de la demo end-to-end (Fase 5) bajo la Opción A. **tfg-sim** decide y verifica; **tfg-ros** sólo navega y devuelve la pose real. El **PoseSyncBridge** reprojecta esa pose al frame de Habitat, donde YOLOE confirma sobre los objetos colocados. Si no se confirma, la política elige otro candidato y se repite el ciclo. ROS nunca percibe.

vóxel queda asignado sólo a la categoría que gana el **argmax** frente al resto del vocabulario y la clase de fondo **other**.

Además de esta validación, se dejan documentados los arreglos que hicieron posible el flujo completo: captura robusta ante fallos TF con fallback a `/odom`, carga correcta del directorio del run en `VLMa.p.load_map`, `PYTHONPATH` adecuado en el menú, uso de `++` en claves Hydra ya existentes, inyección de `params.gs` y `params.cs` al cargar la configuración aislada y eliminación de la inicialización vacía de categorías.

Mapa evaluado	Celdas ocupadas	IoU medio	IoU máximo	Par máximo
small_house ROS/Gazebo	25 629	0.00010	0.00126	refrigerator-sink
HSSD 102344280_0	21 453	0.0120	0.1655	table-chair

Cuadro 5: Resultado del índice semántico multiclase para categorías de habitaciones distintas. En el mapa capturado desde ROS/Gazebo las máscaras son prácticamente disjuntas; en HSSD el mayor solape aparece entre mesa y silla, que es una proximidad semánticamente esperable.

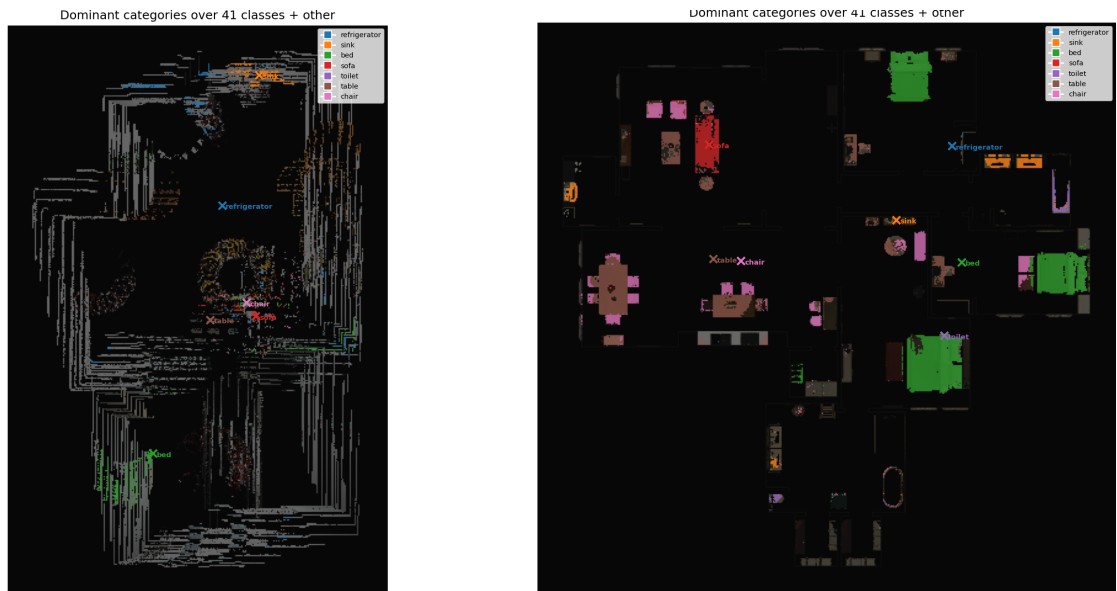


Figura 16: Mapa de categoría dominante por celda para la captura **small_house** (izquierda) y para HSSD 102344280_0 (derecha). La captura ROS/Gazebo separa cama, cocina, zona común y ausencia de baño con un solape medio muy bajo.